ActiveRecord: Banco de Dados em Ruby e em Rails

Bruno Müller Junior

Departamento de Informática

UFPR

2020

Sumário

1 ActiveRecord em Ruby 7
1.1 O básico: Activerecord em uma tabela 9
1.1.1 Tabelas e Modelos 9
1.1.2 Operações 11
1.1.2.1 Inclusão 11
1.1.2.2 Consultas 14
1.1.2.3 Exclusão 15
1.1.2.4 Alterações 15
1.2 Validações 16
1.2.1 Validação com o modificador `validate` 16
1.2.2 Validação com *Helpers* 17
1.2.3 Validação com `validates_with` 17
1.3 *Callbacks* 17
1.4 Relacionamentos 22
1.4.1 belongs_to 23
1.4.1.1 Resumo do modificador `belongs_to` 26
1.4.2 has_one 27
1.4.2.1 Resumo do modificador `has_one` 28
1.4.3 has_many 30
1.4.3.1 Resumo do modificador `has_many` 32
1.4.4 has_and_belongs_to_many 33
1.4.4.1 Resumo do modificador `has_and_belongs_to_many` 36
1.5 Testes 37
2 `ActiveRecord` em Rails 43
2.1 O básico: Activerecord em uma tabela 43
2.1.1 Operações 45
2.2 Validações 47
2.3 *Callbacks* 48
2.4 Relacionamentos 48
2.5 Relacionamento 1:1 48
2.6 Relacionamento 1:N 50
2.7 Relacionamento N:N 52
2.8 Testes 54
2.8.1 O que ocorre com `rake test` 56
2.8.2 O ciclo do programador para testar um modelo 57
2.8.3 Mais sobre testes 60
3 Métodos Frequentemente Utiizados 61
3.1 Métodos de busca 61
3.1.1 all 61
3.1.2 find_by 61

3.1.3 where . . . 61
3.1.4 order . . . 62
3.1.5 select . . . 62
3.1.6 delete . . . 62
3.1.7 delete_all . . . 62
3.2 Métodos de iteração . . . 62
3.2.1 find_each . . . 62
3.2.2 find_in_batches . . . 62
Referências . . . 63

Prefácio

Este texto baseia-se em [1] e foi desenvolvido para orientar os alunos da disciplina de Programação Web a criar e usar um banco de dados utilizando as facilidades propostas pelo framework ActiveRecord.

Os pré-requisitos para o bom entendimento do texto são (1) programação orientada a objetos (2) linguagem Ruby e (3) modelagem e uso de bancos de dados relacionais (em especial SQL).

1. ActiveRecord em Ruby

Linguagens orientadas a objeto e bancos de dados relacionais utilizam estratégias diferentes para armazenar informação. Enquanto em linguagens orientadas a objeto a estrutura básica é o objeto, em bancos de dados, as estruturas básicas são tabelas organizadas em linhas (também conhecidas por tuplas).

Aplicações que usam linguagens orientadas a objeto muitas vezes precisam acessar bancos de dados. Quando o BD é relacional, a solução adotada é mapear as tabelas em classes e cada tupla (ou linha) é mapeada em um objeto sendo que cada atributo da tabela é associada a um atributo de mesmo nome na classe. Este modelo é conhecido na literatura como mapeamento objeto-relacional, a sigla em inglês é "ORM" (*Object-relational mapping*).

Para a linguagem Ruby foi desenvolvido um ORM próprio, que foi implementado com o nome de Active Record. Porém, é importante saber que a implementação segue o padrão proposto em [2]. Sugerimos fortemente a leitura do texto para entender a conceituação de cada aspecto da biblioteca.

Este texto aborda a biblioteca Active Record, e poucos conceitos teóricos.

Para tal vamos apresentar um exemplo de ORM, já com a taxonomia de Active Record, porém algumas coisas ficarão em aberto e serão explicadas posteriormente.

No exemplo, considere a existência de uma tabela com o nome `estados` que contém uma lista de estados brasileiros. Cada linha da tabela contém dois atributos: sigla e nome como indicado na tabela 1.

Tabela estados	
Sigla	**Nome**
AC	Acre
AL	Alagoas
AP	Amapá
AM	Amazonas
...	...

Tabela 1 – Tabela `estados` em um BD relacional

Na linguagem orientada a objetos, a idéia é criar uma classe cujos atributos sejam os mesmos da tabela, no caso, sigla e nome. Em Ruby, seria algo como indicado no algoritmo 1.

```
1 class Estado < ActiveRecord::Base;
2       # attr_accessible :sigla, :nome
3 end
```

Algoritmo 1 : Classe estado

Alguns pontos importantes:

- A linha 1 indica que a classe `Estado` é derivada da classe `ActiveRecord::Base`. Isto significa que um objeto instanciado pela classe `Estado` herdará os métodos disponíveis na classe

`ActiveRecord::Base`. Estes métodos foram projetados para tornar intuitivo o acesso às tabelas do banco de dados, como por exemplo, o método `find`.

- A linha 2 indica os nomes dos atributos dentro da classe. Observe que está comentado, uma vez que o ActiveRecord sabe o nome deles pelos nomes dos atributos na tabela.

Os comandos abaixo é um exemplo de como acessar a tabela:

```
> est = Estado.find(:sigla => "AM")
> puts est.nome
Amazonas
```

A tabela estado (tabela 1) está armazenada num SGBD relacional. Esta tabela está associada à classe "Estado"como indicado no algoritmo 1 (ainda falta explicar como).

Assim, o comando `Estado.find(:sigla => "AM")` diz para procurar na classe Estado (ou seja, na tabela estado), a tupla que contém a sigla `"AM"` e criar um novo objeto (`est`) contendo o resultado da busca.

Se não encontrar uma tupla que atenda ao critério de busca, então será retornado `NullClass` para `est`. Porém, se encontrar, então o novo objeto `est` será preenchido com os dados da tupla, ou seja, `est.nome = "Amazonas"` e `est.sigla = "AM"`.

Observe que a linha 2 do algoritmo 1 está comentada. Parece um erro, pois sem ela como seria possível saber o nome dos atributos?

Isto funciona porque o `ActiveRecord` exige padronização de nomes, e com isto ele pode "deduzir" muita coisa. Por causa desta padronização de nomes, o `ActiveRecord` sabe que a classe Estado (primeira letra em maiúsculo, no singular) está associado à tabela estados (no plural, em minúsculo). Em seguida, ele procura esta tabela no SGBD e, se encontrá-la, então cada atributo da tabela vira um atributo da classe "automagicamente"[1].

1.1 O básico: Activerecord em uma tabela

Esta seção apresenta os conceitos básicos para a utilização do ActiveRecord em Ruby. O termo "básicos" indica operações sobre uma única tabela.

A seção 1.1.1 descreve como criar a tabela em seu respectivo modelo. A seção 1.1.2 descreve algumas das operações que podem ser realizadas sobre uma tabela e os métodos Activerecord que os implementam.

1.1.1 Tabelas e Modelos

O algoritmo 2 descreve como criar o banco de dados "Tabelas.sqlite3"que contém uma única tabela, chamada `"estados"`. Esta tabela é composta por dois atributos: sigla (um *string* de dois caracteres) e nome (*string*).

Vamos examinar o arquivo com mais detalhes:

linha 1 e 2: Inclui as bibliotecas (arquivos) necessários.

[1] Ruby é uma linguagem que permite a inclusão de atributos em uma classe em tempo de execução. Esta é uma das funcionalidades daquilo que se chama "metaprogramação".

```
1  require 'rubygems'
2  require 'active_record'
3
4  ActiveRecord::Base.establish_connection :adapter => "sqlite3",
5                                          :database => "Tabelas.sqlite3"
6
7  ActiveRecord::Base.connection.create_table :estados do |t|
8      t.string :sigla, limit: 2
9      t.string :nome
10 end
```

Algoritmo 2 : Arquivo "criaEstados.rb"

linhas 4 a 5: Indica os parâmetros do BD a ser criado. No caso, com SGBD `sqlite3` e arquivo com nome `Tabelas.sqlite3`. Usamos o SGBD mais simples, e propício para aulas, o `sqlite3`. Outras possibilidades incluem Postgres e MySql.

linhas 7 a 10: Indica o nome da tabela a ser criada (`estados`) os tipos e nomes de seus atributos. No exemplo, os atributos são de um único tipo, `string`, mas existem muitos outros.

Ao ser executado, será criado o arquivo `"Tabelas.sqlite3"`.

```
> ls
criaEstados.rb
> ruby criaEstados.rb
> ls
Tabelas.sqlite3  criaEstados.rb
```

Este arquivo contém a tabela criada, e a mesma pode ser consultada pelo programa sqlite3 (com comandos SQL) conforme descrito abaixo:

```
> sqlite3 Tabelas.sqlite3
SQLite version 3.8.2 2013-12-06 14:53:30
Enter ".help" for instructions
Enter SQL statements terminated with a ";"
sqlite> .tables
estados
sqlite> .schema
CREATE TABLE "estados" ("id" INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL, "sigla" varchar(2), "nome" varchar);
sqlite>
```

O último comando digitado (`.schema`) faz o SGBD imprimir as tabelas e os atributos de cada uma. Isto permite ver que a tabela `estados` tem três atributos:

`id` chave primária (um inteiro) criada automaticamente;

`sigla` atributo com dois caracteres (varchar(2));

`nome` atributo com número variado de caracteres;

Os atributos `sigla` e `nome` foram indicados no algoritmo 2, mas o atributo `id` não foi. Normalmente cada tabela criada com Activerecord inclui um atributo numérico, criado automaticamente, com o nome `id` que contém a chave primária daquela tabela. É possível mudar isso,

por exemplo indicando que não há chave primária (como será visto em 1.4.4) ou que a chave primária é de outro tipo (o que não será apresentado neste texto).

O segundo passo é criar uma classe que permita acesso à tabela. Em Ruby, recomenda-se fortemente a criação de um arquivo de classe para cada tabela. Assim, se a aplicação usar oito tabelas, então deverão existir oito arquivos, cada um contendo exatamente uma classe.

Além disso, sugere-se que o nome do arquivo seja o mesmo do nome classe, mas em minúsculas. Seguindo este padrão, chega-se ao algoritmo 3.

```
1 require 'active_record'
2
3 ActiveRecord::Base.establish_connection :adapter => "sqlite3",
4                                         :database => "Tabelas.sqlite3"
5
6 class Estado < ActiveRecord::Base;
7 end
```

Algoritmo 3 : Arquivo "estado.rb"

As linhas da conexão são iguais às apresentadas no algoritmo 2, e indicam o nome do banco de dados que contém as tabelas. As linhas 6 e 7 criam a classe estado, dizendo que ela herda suas características de `ActiveRecord::Base`. Baseado nesta informação, o Activerecord procura, no banco de dados `"Tabelas.sqlite3"`, a tabela com o nome `"estados"`. O nome da tabela é obtido ao pluralizar o nome da classe e colocar todos os caracteres em minúsculo. Isto é padrão do Activerecord.

Ao final desta tarefa, serão inseridos, no objeto, os mesmos atributos existentes na tabela.

1.1.2 Operações

Com a tabela e modelo criados, o próximo passo é executar operações inclusão e consulta de dados na tabela.

Isto será feito em seções separadas: inclusão na seção 1.1.2.1, impressão na seção 1.1.2.2, exclusão na seção 1.1.2.3 e alteração na seção 1.1.2.4.

Os exemplos poderiam ser colocados em um único arquivo, mas para simplificar a explicação, serão criados vários - um por operação.

1.1.2.1 Inclusão

Para incluir dados em uma tabela, primeiro é necessário:

1. incluir o modelo;
2. instanciar um objeto;
3. inserir valores nos atributos;
4. salvar o objeto.

Estes passos estão no arquivo `"populaEstados.rb"` (algoritmo 4).

O primeiro passo é incluir o modelo. O arquivo contendo o modelo (que é contém o código para fazer a conexão com o banco de dados) é incluído na linha 2. Observe a linha 1 inclui um o diretório onde está o modelo da linha 2. No caso, o diretório incluído é o corrente.

```
1  $:.push '.'
2  require 'estado.rb'
3   # Método 1
4       est = Estado.new ()
5       est.nome = "Acre"
6       est.sigla = "AC"
7       est.save
8
9  # Método 2
10      est = Estado.new ({:nome => "Alagoas", :sigla => "AL"})
11      est.save
12
13 # Método 3
14      lista_estados = [
15          {:nome => "Bahia", :sigla => "BA"},
16          {:nome => "Ceará", :sigla => "CE"},
17          ...
18          {:nome => "Sergipe", :sigla => "SE"},
19          {:nome => "Tocantins", :sigla => "TO"},
20      ]
21      lista_estados.each do |e|
22          est = Estado.new ()
23          est.nome = e[:nome]
24          est.sigla = e[:sigla]
25          est.save
26      end
```

Algoritmo 4 : Arquivo "populaEstados.rb"

O segundo passo é instanciar um objeto, o que é feito em várias linhas, como por exemplo a linha 4.

O terceiro passo é inserir valores nos atributos. Existem várias formas de fazê-lo:

Método 1 Aqui, primeiro cria-se um novo objeto da classe Estado. O objeto `est` é a instancia da classe e tem dois atributos (`nome` e `sigla`). As atribuições que se seguem inserem valores no objeto, porém estes só serão incluídos na tabela `"estados"` do banco de dados após execução do método `save`. Os métodos `new` e `save` são herdados de `ActiveRecord`, além de muitos outros.

Método 2 Aqui utiliza-se passagem de parâmetro por nome[2] utilizando uma hash contendo os parâmetros e os valores.

Método 3 Entre as linhas 14 e 20, são colocadas várias entradas hash em um array. Entre as linhas 21 e 21, há um laço de criação e salvamento dos registros do vetor.

Como o arquivo é um programa Ruby, é possível inserir elementos de outras fontes como por exemplo a partir de arquivos CSV, de planilhas, etc.

O método `save` é um dos métodos herdados da classe ActiveRecord. É insteressante observar como ele funciona.

[2] também conhecido como `keyword parameters`

Ele é um método Ruby que gera um (ou mais) comandos SQL utilizando os atributos do objeto (e consequentemente da tabela) como parâmetro. No caso, ele irá gerar uma sequencia SQL para gravar o conteúdo do objeto na tabela. Algo como o indicado abaixo:

```
insert into "estados" ("nome", "sigla") VALUES ("Acre", "AC");
```

Além do método `save`, o Activerecord contém vários outros métodos para gravação, mas não serão abordados aqui.

Após a execução do arquivo (`ruby populaEstados.rb`), é possível ver os dados armazenados no banco de dados (algumas linhas foram suprimidas).

```
$ sqlite3 Tabelas.sqlite3
SQLite version 3.8.2 2013-12-06 14:53:30
Enter ".help" for instructions
Enter SQL statements terminated with a ";"
sqlite> select * from estados;
1|AC|Acre
2|AL|Alagoas
3|AP|Amapá
4|AM|Amazonas
5|BA|Bahia
...
28|TO|Tocantins
sqlite>
```

1.1.2.2 Consultas

Esta seção explica como recuperar dados salvos em uma tabela. Para tal, criamos o arquivo `listaEstados.rb` apresentado no algoritmo 5.

Este arquivo apresenta `ALGUMAS` das várias formas de acessar uma tabela usando métodos herdados da classe `ActiveRecord`.

Os primeiros três métodos acessam um único registro.

Método 1.1 O método `find` permite acesso pela chave primária.

Método 1.2 O método `find_by` procura o primeiro registro que atende aos critérios da hash passada como parâmetro.

Método 1.3 O método `find_by_sigla` é um método criado pelo `ActiveRecord` especificamente para a classe Estados. Ele cria um destes por atributo (ou seja, também é criado o `find_by_nome`. A metaprogramação é que permite a criação deste tipo de métodos, chamados *dynamic finders*.

Os dois últimos mecanismos são para recuperar vários registros de uma só vez.

Método 2.1 O método `find_by_sql` permite recuperar todos os registros que atendem ao comando SQL passado no parâmetro. O objeto resultante é um array contendo os registros que atendem à consulta.

Método 2.2 O método `all` recupera TODOS os registros da tabela no objeto especificado. Observe que o objeto resultante é da classe `ActiveRecord_Relation`, diferente do resultado do método 2.1 (`find_by_sql`).

Por fim, as linhas 24 até 26 listam o resultado da consulta. Ele pode ser usado tanto com o método 2.1 quanto com o método 2.2.

É importante lembrar que os métodos de consulta acima são mapeados em comandos SQL. Por exemplo, `est = Estado.all` é mapeado para um comando SQL como `select * from estados`, e o resultado é mapeado em um `ActiveRecord_Relation`. Em outras palavras, o Activerecord faz a conversão de objetos em tabelas a vice-versa normalmente usando SQL.

```
1  $:.push './'
2  require 'estado.rb'
3
4  # Método 1.1: usando a chave primária
5  est = Estado.find(1)
6  puts est.nome
7
8  # Método 1.2: fazendo consulta com busca por nome
9  est = Estado.find_by ({:sigla => "AL"})
10 puts est.nome
11
12 # Método 1.3: fazendo consulta usando dynamic finders
13 est = Estado.find_by_sigla("AM")
14 puts est.nome
15
16 # Método 2.1: usando sql
17 est = Estado.find_by_sql("SELECT * from estados")
18 puts est.class # Array
19
20 # Método 2.2: pegando todos
21 est = Estado.all
22 puts est.class # Estado::ActiveRecord_Relation
23
24 est.each do |e|
25      puts "#{e.sigla} = #{e.nome}"
26 end
```

Algoritmo 5 : Arquivo "listaEstados.rb"

1.1.2.3 Exclusão

Existem dois métodos para exclusão de tuplas em bancos de dados: `delete` e `destroy`. A diferença entre eles é que `delete` exclui o registro independente do usuário colocar validações específicas ou *callbacks*.

Nos dois casos o procedimento é o mesmo:

1. recupera o registro (por exemplo `est=Estado.find(1)`);
2. executa o método de exclusão desejado (por exemplo `est.delete`);

Existem também métodos de exclusão de todos os registros que atendem a um critério como `Estado.delete_all{sigla: "AM"}`.

1.1.2.4 Alterações

Uma forma simples de executar alterações é:

1. encontrar registro que atende a um critério (por exemplo `.find`) e instanciar um objeto com o registro;
2. alterar o valor do atributo desejado;
3. salvar o objeto (por exemplo `.save`);

Este mecanismo funciona pois o registro encontrado já vem com a chave primária indicada, e o `.save` sobreescreve aquele registro.

Uma alternativa é utilizar o método `.update` cujo parâmetro é uma `hash` contendo o atributo a ser modificado e o novo valor.

O algoritmo 6 apresenta exemplos de como alterar um registro utilizando o método `update` e uma `hash` (linhas 9 a 10), como alterar com o método `save` (linhas 12 a 14) e como excluir com o método `destroy` (linha 17).

```
1  $:.push './'
2  require 'estado.rb'
3
4  est = Estado.new ()
5  est.nome = "NAO EXISTE"
6  est.sigla = "NE"
7  est.save
8
9  est = Estado.find_by_sigla("NE")
10 est.update({:sigla => "NN"})
11
12 est = Estado.find_by_sigla("NN")
13 est.sigla = "XX"
14 est.save
15
16 est = Estado.find_by_sigla("XX")
17 est.destroy
```

Algoritmo 6 : Arquivo "alteraEstados.rb"

1.2 Validações

No modelo MVC, o Model é responsável não só pela gravação de um registro, mas também por verificar se os atributos do registro a ser gravado estão em formato consistente. A esta verificação se dá o nome de validação.

Como exemplo, considere um objeto da classe `Estado`. Os seus atributos, `sigla` e `nome`, devem estar num formato onde:

- cada um só pode ser composto de letras;
- não podem ser vazios ();
- sigla tem de ter exatamente duas letras maiúsculas;

Estas validações podem ser feitas direto no código, porém é boa prática fazê-las (ou indicá-las) no próprio modelo. Existem várias abordagens para tal, mas vamos nos concentrar em algumas poucas.

1.2.1 Validação com o modificador `validate`

O modificador `validate` indica os métodos (definidos pelo usuário) a serem executados quando um objeto for verificado com `.valid?` ou `.invalid?`.

O algoritmo 7 exemplifica o uso.

O código entre as linhas 3 e 7 abre a classe String e inclui um método local (`only_letters?`) que verifica se o string só contém letras.

A definição da classe Estado é semelhante ao que já foi apresentado. As novidades são:

linha 14 Aqui o modificador `validate` é incluído e os parâmetros são os métodos a serem executados quando o objeto executar o método `.invalid?` (linha 31).

Línhas 16 e 21 a implementação dos métodos para validação. Observe as linhas 18 e 23. Caso seja detectado um erro, a mensagem de erro será incluída no símbolo `:sigla`, permitindo a impressão de erros por atributo.

Línha 30 insere uma sigla inválida;

linha 31 método que testa se o objeto está num formato válido ou não. O teste consiste em executar os métodos indicados no `validate` da classs Estado.

linha 33 imprime os erros em `:sigla`.

```
1  require 'active_record'
2
3  class String
4      def only_letters?
5          !!match(/^[[:alpha:]]+$/)
6      end
7  end
8
9  ActiveRecord::Base.establish_connection :adapter => "sqlite3",
10                                          :database => "Tabelas.sqlite3"
11
12
13 class Estado < ActiveRecord::Base;
14     validate :sigla_tem_tamanho_dois, :sigla_somente_caracteres
15
16     def sigla_tem_tamanho_dois
17         if sigla.size != 2
18             errors.add(:sigla, "Siga tem de ter tamanho dois")
19         end
20     end
21     def sigla_somente_caracteres
22         if !sigla.only_letters?
23             errors.add(:sigla, "Sigla pode ter somente caracteres")
24         end
25     end
26 end
27
28 est = Estado.new ()
29 est.nome = "x"
30 est.sigla = "1"
31 if est.invalid?
32     puts "Registro Invalido:"
33     est.errors[:sigla].each do |s|
34         puts "   #{s}"
35     end
36 end
37
```

Algoritmo 7 : Validação utilizando `validate`

1.2.2 Validação com *Helpers*

Helpers são métodos pré-definidos para executar determinadas tarefas. No presente contexto, falamos de *validation helpers* do ActiveRecord.

Cada *helper* é transformado em um método como os apresentados no algoritmo 7. Para dar uma idéia mais clara, reescrevemos o algoritmo 7 com `helpers` equivalentes no algoritmo 8.

```
1  require 'active_record'
2
3  ActiveRecord::Base.establish_connection :adapter => "sqlite3",
4                                          :database => "Tabelas.sqlite3"
5
6  class Estado < ActiveRecord::Base;
7      validates :sigla, format:
8          { with: /\A [a-zA-Z]+ z/, message: "Sigla pode ter somente caracteres"}
9      validates :sigla, length:
10         { is: 2, message: "Siga tem de ter tamanho dois"}
11 end
12
13 est = Estado.new ()
14 est.nome = "x"
15 est.sigla = "1"
16 if est.invalid?
17     puts "Registro Invalido:"
18     est.errors[:sigla].each do |s|
19         puts "#{s}"
20     end
21 end
```

Algoritmo 8 : Validação utilizando `helpers`

Os dois *helpers* utilizados foram:

format Diz para analisar o atributo `:sigla` com os parâmetros indicados no `:format`. O parâmetro `with:` contém a expressão regular a ser utilizada e o parâmetro `message:` indica a mensagem de erro a ser colocada em `errors`;

lenght Diz para analisar o atributo `:sigla` com os parâmetros indicados no `:lenght`. O parâmetro `is:` diz que tem de ter exatamente dois (outros são `minimum:`,`maximum:` e `in:`).

Como já foi indicado, existem dezenas de `validate helpers`. Para mais *helpers*, consulte `http://guides.rubyonrails.org/active_record_validations.html`

1.2.3 Validação com `validates_with`

O modificador `validates` pode ser utilizado em várias situações onde o usuário quer criar validadores próprios. Porém, existe uma solução diferente que consiste em criar uma classe específica para a validação.

O arquivo arquivo `"estadoClassVW"` (algoritmo 9) exemplifica a validação com `validates_with`.

A linha 23 contém o modificador `validates_with`, cujo parâmetro indica o nome da classe (derivada de `ActiveModel::Validator`) que irá fazer a validação.

Neste exemplo, a classe `EstadoValidador` está no mesmo arquivo do modelo, porém o mais natural é que esteja em um arquivo diferente.

O método de entrada nesta classe é `validate(record)` (linha 12). O parâmetro `record` contém os atributos da classe `Estado`, em especial `sigla`. Observe como acessar o atributo: `record.sigla`.

O resto do código já foi explicado anteriormente.

1.3 *Callbacks*

`http://api.rubyonrails.org/classes/ActiveRecord/Callbacks.html`

```
1  require 'active_record'
2
3  class String
4      def only_letters?
5          !!match(/^[[:alpha:]]+$/)
6      end
7  end
8   ActiveRecord::Base.establish_connection :adapter => "sqlite3",
9                                           :database => "Tabelas.sqlite3"
10
11 class EstadoValidator < ActiveModel::Validator
12     def validate(record)
13         if (record.sigla.size != 2)
14             record.errors.add(:sigla, "Siga tem de ter tamanho dois")
15         end
16         if !record.sigla.alpha?
17             record.errors.add(:sigla, "Sigla pode ter somente caracteres")
18         end
19     end
20 end
21
22 class Estado < ActiveRecord::Base;
23     validates_with EstadoValidator
24 end
25
26 est = Estado.new ()
27 est.nome = "x"
28 est.sigla = "1"
29 if est.invalid?
30     puts "Registro Invalido:"
31     est.errors[:sigla].each do |s|
32         puts "#{s}"
33     end
34 end
```

Algoritmo 9 : Validação utilizando `validates_with` (arquivo `"estadoClassVW"`)

`Callbacks` são métodos invocados antes ou depois de alguma alteração no estado do objeto. Existem vários momentos no ciclo de vida de um objeto em que seu estado é alterado, como por exemplo na criação, alteração e exclusão do objeto. No caso de objetos derivados de ActiveRecord, estes momentos incluem criação, escrita e leitura no banco de dados, além da validação da exclusão entre outros.

Esta seção trata exclusivamente dos `Callbacks` de objetos derivados da classe ActiveRecord.

Considere o que ocorre quando o método `.save` é executado. A primeira impressão é que a única ação é o objeto ser mapeado como uma tupla na tabela correspondente, salvo, e o controle de execução é passado à linha seguinte ao método `.save`.

Porém, entre estes três pontos, o ActiveRecord disponibiliza uma série de métodos intermediários, denomindados *Callbacks*. O mesmo ocorre entre o método `.find`, a leitura da tabela e o retorno ao fluxo de execução.

As vantagens desta abordagem podem ser melhor explicada com um exemplo. Suponha que um projeto exige que os dados armazenados no banco de dados devem estar criptografados. Isto significa que o programador deve criptografar cada objeto antes dele ser salvo no BD e deve decriptografar cada tupla lida antes de passá-la ao objeto.

Existem várias formas de implementar isto, como por exemplo criar funções invólucro[3] na classe. Estas funções são os responsáveis pelas tarefas acima de tal forma que o processo seja isolado dos demais módulos do projeto.

[3] wrapper functions

Como ActiveRecord inclui funções de `callback`, uma outra solução é implementar algumas delas na classe. Evidentemente, o primeiro passo para tal é saber quais são as funções de `callback`, o que é público e disponíveis em vários sítios da Web como por exemplo `http://guides.rubyonrails.org/active_record_callbacks.html#available-callbacks`. Este link apresenta não só os disponíveis no ActiveRecord, que são os seguintes (na sequencia):

* save (invocação do método do método save)

* valid

1 before_validation

* validate

2 after_validation

3 before_save

4 before_create

* create (salvamento na tabela)

5 after_create

6 after_save

7 after_commit

A sequência de callbacks acima está indicada com números. As linhas indicadas com asteriscos indicam algum tipo de ação. Assim, ao encontrar o método `.save` (ou `.valid`), o primeiro *callback* executado é `before_validation` e ao terminar a execução, os métodos associados à validação são disparados. Os demais passos tem nomes auto-explicativos.

Observe o método `.create` (entre o quarto e o quinto `callback`). É neste ponto que o objeto é salvo na tabela. Observe que esta hierarquia também demonstra a diferença entre o método `.save` e o método `.create`: o primeiro passa por vários `callbacks` e pela validação enquanto que o segundo vai direto para o quarto callback e retorna após o quinto.

Além dos `callbacks` relacionados com o salvamento de um registro, existem outros relacionados aos objetos em geral, como a instanciação (`before_initialize`, `after_initialize`) e liberação (`before_destroy`, `after_destroy`).

No contexto de ActiveRecord, dois outros *callbacks* são muito utilizados: `before_find` e `after_find`. Um exemplo de como utilizar as `callbacks` é apresentado no arquivo `"estadoCB.rb"` (algoritmo 10.

lin:befSaveD lin:aftFindD lin:befSaveI lin:aftFindI

Existem várias formas de se implementar `callbacks`. O algoritmo 10 utiliza a referência ao método do `callback`[4], uma das duas formas recomendadas[5].

As linhas 7 e 8 indicam o nome do *callback* e o nome do método que deve ser executado quando aquele `callback` ocorrer.

As linhas 11 e 14 contém os métodos de implementação declarados como `private` (linha 10).

É interessante observar o que ocorre ao invocar o método `.all` na linha 23: o método `apos_o_find` será invocado uma vez por registro lido.

1.4 Relacionamentos

Quando se cria um banco de dados relacional, utiliza-se uma série de passos que permitem (1) armazenamento consistente e (2) acesso eficiente aos dados. Esses passos reduzem a redundância de dados e as chances dos dados se tornarem inconsistentes.

A série de passos chama-se "normalização de dados". Assumimos que o leitor já conhece estes passos, que são chamadas "formas normais", e que o leitor entende que as tabelas que serão mostradas daqui para frente já foram normalizadas.

Se o leitor não conhece normalização ou os termos "chave primária", "chave estrangeira", "relação 1:1", "relação 1:N" e "relação N:N", sugerimos fortemente ler algum livro que os explica.

Ao criar uma aplicação o programador mais ingênuo nem sempre pensa na normalização. Alguns entendem que ela é desnecessária e outros ainda pensam que o próprio SGBD (ou o `ActiveRecord`) faz a normalização. Isto é falso.

A normalização é imprescindível num bom projeto de banco de dados e tem de ser pensada cuidadosamente **ANTES DE INICIAR** o desenvolvimento da aplicação.

[4] method reference callbacks

[5] a outra é chamada *callback objects* (veja em `http://api.rubyonrails.org/classes/ActiveRecord/Callbacks.html`)

```
1  require 'active_record'
2
3  ActiveRecord::Base.establish_connection :adapter => "sqlite3",
4                                          :database => "Tabelas.sqlite3"
5
6  class Estado < ActiveRecord::Base;
7      before_save :antes_do_save
8      after_find :apos_o_find
9
10 private:
11     def antes_do_save
12         puts "ANTES DO SAVE"
13     end
14     def apos_o_find
15         puts "DEPOIS DE FIND #sigla #nome"
16     end
17 end
18
19 est = Estado.new ()
20 est.nome = "x"
21 est.sigla = "1"
22
23 est = Estado.all
24 est.each do |e|
25     puts "#e.sigla #e.nome"
26 end
```

Algoritmo 10 : Arquivo arquivo `"estadoCB"`

Após o projeto do BD estar pronto, aí sim chega o passo de implementar as tabelas e os relacionamentos advindos da aplicação das formas normais.

Destacamos isso tão fortemente pois muitos livros de Ruby e de Rails descrevem os relacionamentos que o `ActiveRecord` proporciona como se fossem tópicos mágicos, sem muito contexto. Por esta razão, muitos leitores se perdem na explicação ou pior, pensam ser uma questão menor no projeto de um aplicativo.

O `ActiveRecord` inclui várias facilidades para implementar os relacionamentos, e estas facilidades são usadas em conjunto para implementar as relações canônicas de BD (1:1, 1:N e N:N).

Estas facilidades incluem métodos aos objetos das classes que herdam de `ActiveRecord`. São elas: belongs_to, has_one, has_many, has_one :through e has_many :through. Daqui para frente, chamaremos estas facilidades de "modificadores".

As próximas seções explicam a semântica dos modificadores belongs_to (seção 1.4.1), has_one (seção 1.4.2) e has_many (seção 1.4.3). Estes três modificadores são suficientes para implementar as relações 1:1, 1:n e n:n.

Os modificadores has_one :through e has_many :through não serão explicados aqui.

1.4.1 belongs_to

A semântica do modificador `belongs_to` diz que na tabela que ele for incluído, haverá uma chave estrangeira para outra tabela.

Para exemplificar este modificador, considere uma tabela pessoas, cujos atributos são nome, endereço, cidade, sigla_estado e nome_estado.

Um dos objetivos da normalização é melhorar a qualidade da informação. Ao utilizar os atributos sigla_estado e nome_estado, existe a possibilidade de entrada de dados inválida, como por exemplo "EX" "Praná".

Por esta razão, é interessante criar uma tabela de estados contendo a sigla e o nome do estado (como já foi feito nas seções anteriores) e colocar na tabela de pessoas somente a chave estrangeira para a tabela de estados.

Novamente, o `ActiveRecord` apresenta uma série de facilidades para tal.

O primeiro passo é criar a nova tabela conforme apresentado no arquivo `criaPessoas.rb` (algoritmo 11.

```
1  require 'active_record'
2
3  ActiveRecord::Base.establish_connection :adapter => "sqlite3",
4                                          :database => "Tabelas.sqlite3"
5
6  ActiveRecord::Base.connection.create_table :pessoas do |t|
7      t.string :sobrenome
8      t.string :nome
9      t.string :endereco
10     t.references :estado, foreign_key: true
11 end
```

Algoritmo 11 : Arquivo "criaPessoas.rb"

As linhas 1 até 4 são idênticas ao arquivo `criaEstados.rb` (algoritmo 2). Já o conteúdo das demais linhas são diferentes, mas a maioria do seu conteúdo pode ser compreendido sem explicações.

A excessão é a linha 10, que pede para criar uma referência à tabela de estado (no singular, o que confunde um monte pois a padronização sugeriria estados) indicando que é uma chave estrangeira.

Após criar a tabela, ela pode ser vista com o sqlite3:

```
$ sqlite3 Tabelas.sqlite3
SQLite version 3.8.2 2013-12-06 14:53:30
Enter ".help" for instructions
Enter SQL statements terminated with a ";"
sqlite> .schema
CREATE TABLE "estados" ("id" INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL, "sigla" varchar(2), "nome" varchar);
CREATE TABLE "pessoas" ("id" INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL, "sobrenome" varchar, "nome" varchar,
"endereco" varchar, "estado_id" integer);
sqlite>
```

Observe a tabela `pessoas`. Todos os atributos são como esperado, porém o último é interessante pois o nome dele é `"estado_id"` (sim, este nome segue uma padronização: tabela no singular seguido por _id). Este atributo é o que será utilizado para acessar a tabela de estados.

O segundo passo é criar a classe Pessoa e indicar que há um relacionamento entre `"estado_id"` e a tabela de estados. Isto é feito no arquivo `pessoa.rb` (algoritmo 12) na linha 7. Sim, tudo isso é indicado com o modificador de classe `belongs_to :estado` (novamente, no singular. Parece pegadinha).

```
1  require 'active_record'
2
3  ActiveRecord::Base.establish_connection :adapter => "sqlite3",
4                                          :database => "Tabelas.sqlite3"
5
6  class Pessoa < ActiveRecord::Base;
7      belongs_to :estado
8  end
```

Algoritmo 12 : Arquivo "pessoa.rb"

O próximo passo é incluir dados na tabela de pessoas. Usando o arquivo `populaEstados.rb` (algoritmo 4) como referência, não é difícil determinar como fazê-lo: (1) cria um novo objeto da classe Pessoa (2) atribui valores aos atributos e (3) salva o objeto criado.

Isto descreve bem como proceder para os atributos `sobrenome`, `nome` e `endererço`. Porém não se aplica o mesmo mecanismo para tratar o atributo `estado_id`.

Para apresentar alguns dos vários mecanismos disponíveis, criamos o arquivo `"populaPessoas.rb"` (algoritmo 13).

Os três métodos apresentados são válidos.

```
1  $:.push './'
2  require 'estado.rb'
3
4  # Método 1: inclusão via chave primária
5  p = Pessoa.new ({:sobrenome => "Silva", :nome => "Joao",
6                   :endereco => "Rua 1"})
7  p.estado_id = 1
8  p.save
9
10 # Método 2: inclusão pelo id obtido pela busca na tabela
11 p = Pessoa.new ({:sobrenome => "Silva", :nome => "Jose",
12                   :endereco => "Rua 2"})
13 est = Estado.find_by_sigla("AL")
14 p.estado_id = est.id
15 p.save
16
17 # Método 3: inclusão usando o objeto
18 p = Pessoa.new ({:sobrenome => "Silva", :nome => "Maria",
19                   :endereco => "Rua 3"})
20 est = Estado.find_by_sigla("AM")
21 p.estado = est
22 p.save
```

Algoritmo 13 : Arquivo "populaPessoas.rb"

Método 1 Neste método, o programador sabe qual a chave primária (da tabela `estados`) que corresponde a cada estado. Normalmente não é uma boa solução, pois o código não fica claro.

Método 2 Aqui o programador primeiro procura o estado (no caso pela sigla) e insere a chave primária diretamente no atributo `estado_id`.

Método 3 Semelhante ao método 2, porém a atribuição é diferente. Aqui, o objeto é atribuído a um atributo que não está presente na tabela (`estado`). É um atalho incluído pelo modificador `belongs_to` que torna o código mais legível.

Após a execução do programa `"populaPessoas.rb"`, é possível examinar os registros incluídos no banco de dados:

```
$ sqlite3 Tabelas.sqlite3
SQLite version 3.8.2 2013-12-06 14:53:30
Enter ".help" for instructions
Enter SQL statements terminated with a ";"
sqlite> select * from pessoas;
1|Silva|Joao|Rua 1|1
2|Silva|Jose|Rua 2|2
3|Silva|Maria|Rua 3|4
sqlite> select * from estados where id=1;
1|AC|Acre
sqlite> select * from estados where id=2;
2|AL|Alagoas
sqlite> select * from estados where id=4;
4|AM|Amazonas
sqlite>
```

Observe que o último atributo é a chave primária da tabela de estados. Os três últimos comandos (`select * from ...`) permitem o acesso aos respectivos atributos (id, sigla, nome).

O próximo passo é listar o conteúdo da tabela de pessoas. Novamente o leitor é capaz de deduzir como fazê-lo a partir do programa `"listaEstados"` (algoritmo 5). Novamente a diferença fica no atributo `estado_id`. A idéia não é imprimir só a chave, mas sim o conteúdo da tabela `estados`.

O programa que faz isto é o `listaPessoas.rb` (algoritmo 14).

```
1  $:.push './'
2  require 'estado.rb'
3
4  # Método 1 : buscando a chave primária na outra tabela
5  pessoa = Pessoa.find(1)
6  est = Estado.find(pessoa.estado_id)
7  puts "#{pessoa.nome} #{pessoa.endereco} #{est.sigla} #{est.nome}"
8
9  pessoas = Pessoa.all
10 pessoas.each do |p|
11 puts "#{p.nome} #{p.endereco} #{p.estado.sigla} #{p.estado.nome}"
12 end
```

Algoritmo 14 : Arquivo "listaPessoas.rb"

1.4.1.1 Resumo do modificador `belongs_to`

- A presença do modificador `belongs_to <tabela no singular>` em um modelo pressupõe que o modelo foi criado com um atributo <tabela no singular>_id como `reference`. Este atributo é uma chave estrangeira para a tabela indicada.
- O presença do modificador habilita a utilização de alguns métodos no atributo virtual (sem `_id`) da chave estrangeira:

  setter p.estado = ... (linha 21 do algoritmo 13

  getter puts p.estado.sigla (linha 11 do algoritmo 14.

  igualdade p.estado?(outroEstado), equivalente a (p.estado == outroEstado), mas não exemplificado aqui.

É importante destacar que o modificador `belongs_to` corresponde a uma relação onde um registro de um modelo está associado a exatamente um registro de outro modelo. Porém, isto não implica dizer que a relação é 1:1 e nem 1:n. Para implementá-los, são necessários mais modificadores.

1.4.2 has_one

Assim como o modificador `belongs_to`, o modificador `has_one` é utilizado para implementar uma relação onde uma tupla de uma tabela se relaciona com exatamente uma tupla de outra tabela através de uma chave estrangeira.

A diferença entre as duas é que o modelo onde for inserido o modificador `has_one` **NÃO** contém a chave estrangeira, que está na outra tabela.

Na prática, o ele é utilizado para implementar uma relação 1:1 entre duas tabelas.

Antes de prosseguir com exemplos, vamos analisar um pouco melhor a relação 1:1. Ela diz que cada registro de uma tabela A está relacionada com exata e unicamente um registro da tabela B e que cada registro da tabela B está relacionada com exata e unicamente um registro da tabela A. Não é muito comum encontrar situações onde esta relação é utilizada. Conceitualmente ela é desnecessária pois é mais simples incluir os atributos da tabela A em B (ou o contrário).

A utilização desta relação na prática está mais relacionada à eficiência do acesso à memória e armazenamento em cache. Por vezes é interessante dividir uma tabela, digamos A, com muitos atributos em várias tabelas, digamos B, C e D onde cada uma conté uma parte dos atributos (e onde $A = B \cup C \cup D$ e $B \cap C \cap D = \emptyset$), pois isto normalmente permite que maior número de registros sejam colocados na memória (considerando que nem todas as tabelas são utilizadas o tempo todo).

Vamos agora estender o banco de dados já criado para comportar uma relação 1:1 onde utilizaremos os modificadores `belongs_to` e `has_one`. Para tal, suponha que a tabela de pessoas deve incorporar alguns novos atributos, que são os documentos de RG, CPF e carteira de motorista. Como a tabela é pequena, a melhor maneira de incluir estes atributos

é na própria tabela. Porém, com o objetivo de exemplificar a relação 1:1 e os modificadores `belongs_to` e `has_one`, vamos criar uma nova tabela, chamada `documentos` para tal.

Uma pessos tem um (`has_one`) conjunto de documentos e cada documento pertence (`belongs_to`) a um pessoa. Desta forma, as tabelas `documentos` e `pessoa` implementam uma relação 1:1.

O próximo passo é adaptar o banco de dados. Como já destacado, o modificador `belongs_to` exige a presença de um atributo que será a chave estrangeira da outra tabela, ou seja, a presença do atributo `pessoa_id` na tabela `documentos`. Com isto em mente foi gerado o arquivo `"criaDocumentos.rb"` (algoritmo 15).

```
1  require 'active_record'
2
3  ActiveRecord::Base.establish_connection :adapter => "sqlite3",
4                                          :database => "Tabelas.sqlite3"
5
6  ActiveRecord::Base.connection.create_table :documentos do |t|
7      t.string :rg
8      t.string :cpf
9      t.string :motorista
10     t.references :pessoa, foreign_key: true
11 end
```

Algoritmo 15 : Arquivo "criaDocumentos.rb"

O outro arquivo a ser criado é `"documento.rb"` (algoritmo 16).

```
1  require 'active_record'
2
3  ActiveRecord::Base.establish_connection :adapter => "sqlite3",
4                                          :database => "Tabelas.sqlite3"
5
6  class Documento < ActiveRecord::Base;
7      belongs_to :pessoa
8  end
```

Algoritmo 16 : Arquivo "documento.rb"

Por fim, vamos inserir registros. Antes, porém, é preciso recriar a tabela de pessoas (pois agora tem um atributo a mais). Aproveitamos e recriamos tudo:

```
> rm -f Tabelas.sqlite3
> ruby criaEstados.rb
> ruby populaEstados.rb
> ruby criaDocumentos.rb
```

Como a tabela de documentos e de pessoas estão intimamente relacionadas, não serão criados dois arquivos separados de criação, mas um só: `"populaPessoasD.rb"` (algoritmo 17).

Há alguns pontos a destacar:

linha 9 a forma de incluir a chave estrangeira na tabela de `documentos` já foi descrita na seção 1.4.1.1.

linhas 10 e 11 Há um problema aqui pois a semântica da relação 1:1 diz que os dois registros são salvos ou nenhum. Se alguma das operações não tiver sucesso, o BD fica inconsistente. Para resolver este problema, utiliza-se transações, que não serão explicadas neste momento.

O próximo passo é acessar os atributos da tabela `documentos` a partir de um registro em `pessoas`. Isto é feito na nova versão de `"listaPessoasD.rb"` (algoritmo 18).

Uma alternativa mais longa para acessar as informações na tabela documentos usa o atributo `p.documento_id`.

```
1  $:.push './'
2  require 'pessoa.rb'
3  require 'estado.rb'
4  require 'documento.rb'
5
6  p = Pessoa.new ({:sobrenome => "Silva", :nome => "Joao", :endereco => "Rua 1"})
7  p.estado = Estado.find_by_sigla("AL")
8  doc = Documento.new ({:rg => "RG-1", :cpf => "CPF-1", :motorista =>"Moto-1"})
9  doc.pessoa = p
10 doc.save
11 p.save
12
13 p = Pessoa.new ({:sobrenome => "Silva", :nome => "Jose", :endereco => "Rua 2"})
14 p.estado = Estado.find_by_sigla("AM")
15
16 doc = Documento.new ({:rg => "RG-2", :cpf => "CPF-2", :motorista =>"Moto-2"})
17 doc.pessoa = p
18 doc.save
19
20 p = Pessoa.new ({:sobrenome => "Silva", :nome => "Maria", :endereco => "Rua 3"})
21 p.estado = Estado.find_by_sigla("AM")
22 doc = Documento.new ({:rg => "RG-3", :cpf => "CPF-3", :motorista =>"Moto-3"})
23 doc.pessoa = p
24 doc.save
25
```

Algoritmo 17 : Arquivo "populaPessoas.rb"

```
1  $:.push './'
2  require 'pessoa.rb'
3  require 'estado.rb'
4  require 'documento.rb'
5
6  pessoas = Pessoa.all
7  pessoas.each do |p|
8      print "#{p.nome} #{p.endereco} #{p.estado.sigla} #{p.estado.nome} "
9      print "RG=#{p.documento.rg} CPF=#{p.documento.cpf} "
10     puts "Motorista=#{p.documento.motorista}"
11 end
12
```

Algoritmo 18 : Arquivo "listaPessoasD.rb"

1.4.2.1 Resumo do modificador has_one

- A presença do modificador `has_one <tabela no singular>` no modelo A pressupõe que a tabela usada como parâmetro contém um atributo com o nome `A_id` como `reference`. Este atributo é uma chave estrangeira para a tabela `A`.
- O presença do modificador habilita a utilização de alguns métodos no atributo virtual (sem `_id`):

  setter p.documento.<atr> = ... (linha 17 do algoritmo 17.

  getter ... = p.documento.<atr> (linha 10 do algoritmo 18).

  O modificador `has_one` normalmente é usado em conjunto para implementar a relação é 1:1.

1.4.3 has_many

O modificador `has_many` é utilizado para indicar que cada tupla de uma tabela, digamos *A*, está relacionada com pelo menos *n* tuplas de outra tabela, digamos *B*.

Ela pode ser usada em várias situações, mas vamos exemplificar com um exemplo de relação 1:n. Para tal, iremos acrescentar uma nova tabela, `municípios` associada à tabela de `estados`. Um estado tem vários municípios (`has_many`) enquanto que um município pertence (`belongs_to`) a um estado.

Em princípio, seria possível imaginar que um município tem um (`has_one`) estado. Para explicar porque utiliza-se o par `has_many` e `belongs_to` (e não `has_many`) e (`has_one`), é preciso analisar a conveniência da localização das chaves estrangeiras.

Em nosso exemplo (estados e municípios), vamos comparar o efeito de onde colocar as chaves estrangeiras.

modelo Estado Se as chaves estrangeiras forem colocadas aqui (`id` do município), cada tupla conteria uma lista de tuplas de municípios indicando os municípios associados. Esta lista deveria conter um número variável de entradas.

modelo Municípios Se as chaves estrangeiras forem colocadas aqui (`id` do estado), cada tupla em `municipios` conteria exatamente uma única chave estrangeira contendo o `id` do estado a que pertencem.

Não é muito difícil deduzir que o plano correto é o de colocar as chaves estrangeiras na tabela de municípios. Pelo que vimos até agora, isto é possível ao dizer criar a tabela `municipios` com uma chave estrangeira (algoritmo 19) e a tabela `estados` sem chave estrangeira (algoritmo 2 na página 9 - que não mudou).

```
1  require 'rubygems'
2  require 'active_record'
3
4  ActiveRecord::Base.establish_connection :adapter => "sqlite3",
5                                          :database => "Tabelas.sqlite3"
6
7  ActiveRecord::Base.connection.create_table :municipios do |t|
8      t.string :nome
9      t.belongs_to :estado, foreign_key: true
10 end
```

Algoritmo 19 : Arquivo "criaMunicipios.rb"

Definido o local onde fica a chave estrangeira, vamos à colocação dos modificadores: o modelo Municipios fica com o modificador `belongs_to` (algoritmo 21) enquanto que o modelo Estados fica com o modificador `has_many` (algoritmo 20)

Após criar as tabelas com `criaMunicipios.rb`, o próximo passo é inserir registros. Isto é feito pelo arquivo `"populaMunicipios.rb"` (algoritmo 22).

Neste programa são criados dois municípios para o estado do Paraná. Primeiro, busca o estado (linha 5. Em seguida, cria-se o município e insere o nome. O próximo passo é utilizar o `setter` fornecido pelo modificador `belongs_to` em município (linhas 9 e 13).

O próximo passo é listar os municípios. O arquivo `"listaMunicipios.rb"` (23) lista todos os estados e todos os municípios pertencentes a cada estado.

O destaque deste programa é a linha 8. A linha indica um mecanismo simples para obter todos os municípios de um estado.

```
1 require 'active_record'
2
3 ActiveRecord::Base.establish_connection :adapter => "sqlite3",
4                                         :database => "Tabelas.sqlite3"
5
6 class Estado < ActiveRecord::Base;
7     has_many :municipios
8 end
```

Algoritmo 20 : Arquivo "estado.rb"

```
1 require 'active_record'
2
3 ActiveRecord::Base.establish_connection :adapter => "sqlite3",
4                                         :database => "Tabelas.sqlite3"
5
6 class Municipio < ActiveRecord::Base;
7     belongs_to :estado
8 end
```

Algoritmo 21 : Arquivo "municipio.rb"

```
1 $:.push './'
2 require 'estado.rb'
3 require 'municipio.rb'
4
5 estado_pr = Estado.find_by_sigla("PR")
6
7 municipio = Municipio.new ()
8 municipio.nome = "Abatiá"
9 municipio.estado = estado_pr
10 municipio.save
11 municipio = Municipio.new ()
12 municipio.nome = "Adrianópolis"
13 municipio.estado = estado_pr
14 municipio.save
```

Algoritmo 22 : Arquivo "populaMunicipios.rb"

```
1 $:.push './'
2 require 'estado.rb'
3 require 'municipio.rb'
4
5 estados = Estado.all
6 estados.each do |estado|
7     puts estado.nome
8     municipios = estado.municipios
9     municipios.each do |m|
10        puts "#m.nome"
11    end
12 end
```

Algoritmo 23 : Arquivo "populaMunicipios.rb"

1.4.3.1 Resumo do modificador has_many

- A presença do modificador `has_many <tabela no plural>` (mantendo o programador atento) no modelo A pressupõe que a tabela usada como parâmetro contém um atributo com o nome `A_id` (no caso `estado_id`). Este atributo é uma chave estrangeira para a tabela `A` (`estados`).
- O presença do modificador habilita a utilização de alguns métodos:

  setter `municipio.estado = estado_pr` (linha 9 do algoritmo 22)

  getter `municipios = estado.municipios` (linha 8 do algoritmo 22), que é equivalente a `municipios = Municipio.where(estado_id: estado.id)`.

  exclusão `Estado.municipios.delete` apaga todos as tuplas de um dado estado na tabela de municípios e também a tupla de estado.

  O modificador `has_one` normalmente é usado em conjunto para implementar a relação é 1:1.

1.4.4 has_and_belongs_to_many

O modificador `has_and_belongs_to_many` é utilizado para implementar uma relação `n:n` entre duas tabelas. Esta relação ocorre quando uma tupla da tabela *A* está relacionada com zero ou mais tuplas de uma tabela *B* e uma tupla da tabela *B* está relacionada com zero ou mais tuplas de uma tabela *A*.

A implementação desta relação exige a presença de uma tabela de referência[6] onde cada tupla contém dois atributos: uma chave estrangeira da tabela *A* e uma chave estrangeira da tabela *B*.

A implementação desta tabela em ActiveRecord utiliza o modificador `has_and_belongs_to_many` em ambos os modelos e que a tabela de referência tenha o nome <tabela A no plural>_<tabela B no plural>.

Para exemplificar esta relação vamos criar o modelo `Esporte`, que se relaciona n:n com o modelo `Pessoa`: uma pessoa pode praticar zero ou mais esportes e um esporte pode ser praticado por zero ou mais pessoas.

O primeiro passo é criar a tabela esportes (algoritmo 24), os modelos Esportes e Pessoas (algoritmo 25 e algoritmo 26) e um arquivo para popular a tabela `esportes` (algoritmo 27).

```
1 require 'rubygems'
2 require 'active_record'
3
4 ActiveRecord::Base.establish_connection :adapter => "sqlite3"
5                                         :database => "Tabelas.sqlite3"
6
7 ActiveRecord::Base.connection.create_table :esportes do |t|
8     t.string :nome
9 end
```

Algoritmo 24 : Arquivo "criaEsportes.rb"

```
1 require 'active_record'
2
3 ActiveRecord::Base.establish_connection :adapter => "sqlite3",
4                                         :database => "Tabelas.sqlite3"
5
6 class Esporte < ActiveRecord::Base;
7     has_and_belongs_to_many :pessoas, -> uniq
8 end
```

Algoritmo 25 : Arquivo "esportes.rb"

Observe a linha 9 do algoritmo 26. Ela contém a declaração `uniq`, que diz que tuplas iguais devem ser desconsideradas. Esta diretiva não impede gravação de registros iguais, só que eles devem ser considerados um só.

[6] *cross-reference table* ou `junction table`

```
1  require 'active_record'
2
3  ActiveRecord::Base.establish_connection :adapter => "sqlite3",
4                                          :database => "Tabelas.sqlite3"
5
6  class Pessoa < ActiveRecord::Base;
7      belongs_to :estado
8      has_one :documento
9      has_and_belongs_to_many :esportes, -> uniq
10 end
```

Algoritmo 26 : Arquivo "pessoa.rb"

```
1  $:.push './'
2  require 'esporte.rb'
3
4  esporte = Esporte.new ({ :nome => "futebol"})
5  esporte.save
6  esporte = Esporte.new ({ :nome => "basquete"})
7  esporte.save
8  esporte = Esporte.new ({ :nome => "volei"})
9  esporte.save
10 esporte = Esporte.new ({ :nome => "truco"})
11 esporte.save
```

Algoritmo 27 : Arquivo "populaEsportes.rb"

Com as tabelas `pessoas` e `esportes` criadas (assim como os modelos respectivos), podemos trabalhar na tabela de referência.

O algoritmo 28 contém o arquivo que cria a tabela que relaciona pessoas com esportes. O nome dela não é escolhido pelo usuário mas determinado pelo ActiveRecord, e corresponde aos nomes das duas tabelas no plural separadas pelo símbolo "_", ou seja: `esportes_pessoas`.

Cada tupla desta tabela é composta por duas chave estrangeiras: `esporte_id` e `pessoa_id`, e será criadas sem chave primária (linha 4).

```
1  require 'rubygems'
2  require 'active_record'
3  ActiveRecord::Base.establish_connection :adapter => "sqlite3",
4                                          :database => "Tabelas.sqlite3"
5
6  ActiveRecord::Base.connection.create_table :esportes_pessoas, id: false do |t|
7      t.references :esporte
8      t.references :pessoa
9  end
```

Algoritmo 28 : Arquivo "criaEsportesPessoas.rb"

Agora, vamos abordar a questão de acessar os dados contidos pela relação.

O objeto instanciado pela classe que contém o modificador `has_and_belongs_to_many` recebe alguns métodos para facilitar o acesso. O arquivo `populaEsportesPessoas.rb` (algoritmo 29) inclui tuplas na tabela `esportes_pessoas` de duas formas diferentes.

Entre as linhas 4 e 8, é inserido um mesmo esporte para todos as pessoas enquanto que entre as linhas 10 e 14 é inserida uma mesma pessoa para todos os esportes.

As linhas críticas são as linhas 7 e 13, onde <objeto>.<tabela no plural> recebem o objeto contendo a tupla com que se associar.

```
1  $:.push './'
2  require 'rubygems'
3  require 'active_record'
4  esporte1 = Esporte.find_by_nome("futebol")
5  pessoas = Pessoa.all
6  pessoas.each do |pessoa|
7      pessoa.esportes « esporte1
8  end
9
10 pessoa1 = Pessoa.find_by_nome("Maria")
11 esportes = Esporte.all
12 esportes.each do |esporte|
13     esporte.pessoas « pessoa1
14 end
```

Algoritmo 29 : Arquivo "populaEsportesPessoas.rb"

O efeito desta linha é criar uma nova tupla na tabela `esportes_pessoas` com as chaves primárias de cada tabela. Talvez seja mais claro verificando no sqlite3:

```
> sqlite3 Tabelas.sqlite3
SQLite version 3.8.2 2013-12-06 14:53:30
Enter ".help" for instructions
Enter SQL statements terminated with a ";"
sqlite> select * from pessoas;
1|Silva|Joao|Rua 1|2
2|Silva|Jose|Rua 2|4
3|Silva|Maria|Rua 3|4
sqlite> select * from esportes;
1|futebol
2|basquete
3|volei
4|truco
sqlite> select * from esportes_pessoas;
1|1
1|2
1|3
1|3
2|3
3|3
4|3
```

Observe que há uma duplicação da tupla contendo `esporte_id=1` (futebol) e `pessoa_id=3` (Maria). Reexamine o algoritmo 29 e explique porque.

A visualização do relacionamento no SGBD é útil, porém não é o que o programador quer. O arquivo `"listaEsportesPessoas.rb"` imprime, para cada pessoa, o esporte que pratica e depois, para cada esporte, as pessoas que o praticam.

1.4.4.1 Resumo do modificador `has_and_belongs_to_many`

- O modificador `has_and_belongs_to_many` é utilizado para implementar uma relação n:n entre duas tabelas.
- É necessário criar uma tabela de associação entre as duas, sendo que o nome desta tabela obedece critérios rígidos, assim como o nome de seus atributos (veja algoritmo 28).
- O presença do modificador habilita a utilização de alguns métodos em qualquer modelo:

  setter `esporte.pessoas « pessoa1`

```
1  $:.push './'
2  require 'rubygems'
3  require 'active_record'
4  require 'esporte.rb'
5  require 'pessoa.rb'
6
7  pessoas = Pessoa.all
8  pessoas.each do |pessoa|
9      esportes = pessoa.esportes
10     print "#{pessoa.nome}->"
11     esportes.each do |esporte|
12         print "#{esporte.nome} "
13     end
14     puts
15 end
16
17 esportes = Esporte.all
18 esportes.each do |esporte|
19     pessoas = esporte.pessoas
20     print "#{esporte.nome}->"
21     pessoas.each do |pessoa|
22         print "#{pessoa.nome} "
23     end
24     puts
25 end
```

Algoritmo 30 : Arquivo "listaEsportesPessoas.rb"

getter `pessoas = esporte.pessoas` retorna o conjunto de pessoas que praticam o esporte instanciado.

1.5 Testes

Uma das coisas mais chatas e irritantes do processo de desenvolvimento de uma aplicação é o processo de testes. Nos meus tempos de programador, lembro de utilizar um processo mais ou menos assim:

- Criar lista de tópicos a serem testados em cada possível interação com o usuário. Seja T esta lista, e que T contém n itens.
- i=1
- repita até i==n;
  - aplicar o teste i
  - Se o resultado do teste foi o esperado, passa ao próximo item (i=i+1);
  - Se o resultado do teste não foi o esperado, analisa o código, corrige o erro, e recomeça os testes (i=1);

Não é difícil perceber que é um algoritmo de ordem $O(n^2)$, pois o último passo indica que, ao encontrar um erro e corrigi-lo, deve-se recomeçar os testes do início ($i = 1$). O motivo para recomeçar do início está em ser possível que a correção do item i cause um erro' em algum outro item, digamos j, sendo que j pode ser menor que i.

Para Ruby, foram criados ambientes para automatizar a execução dos testes. Seu funcionamento básico é o seguinte:

1. criar um arquivo contendo uma classe derivada de uma classe de testes. Um arquivo para cada classe.
2. dentro desta classe, criar métodos sendo que cada método contém o código de um dos n testes a serem aplicados.
3. ao executar o arquivo de teste, todos os métodos de teste, de 1 até n, serão executados. Caso ocorra um erro, será impressa uma mensagem indicando qual método gerou o erro.

Este texto não é um tutorial para utilização de testes em Ruby. Existem vários tutoriais excelentes na Web, e sugiro a leitura do tutorial em `https://en.wikibooks.org/wiki/Ruby_Programming/Unit_testing` antes de prosseguir.

Para exemplificar como criar um arquivo de testes e o seu efeito, vamos utilizar o modelo `estados`. Para simplificar, vamos utilizar a versão mais simples, reproduzida no algoritmo 31.

```
1 class Estado < ActiveRecord::Base;
2       # attr_accessible :sigla, :nome
3 end
```

Algoritmo 31 : Arquivo `"estadoSemValida"`, classe estado sem validação

Esta versão não utiliza nenhuma validação. O plano é criar um arquivo de teste que procura detectar se o modelo permite a gravação de registros onde:

1. sigla está vazia (`nil`).
2. nome está vazio (`nil`).
3. nome contém pelo menos um símbolo especial.
4. nome contém caracteres normais.
5. nome contém caracteres UTF-8.

Existem vários outros testes necessários, mas o nosso exemplo resume-se a estes. O leitor deve implementar outros como “sigla só tem caracteres”, “sigla tem tamanho exatamente dois”, etc.

Nos testes listados acima, a idéia é simples: criar um estado, preencher os atributos com valores que permitam averiguar os testes propostos e verificar se o modelo permite a gravação ou não do registro. Dependendo do resultado, a execução indicará o erro.

O algoritmo 32 implementa estes testes.

A linha 1 inclui o arquivo contendo o modelo enquanto que a linha 2 inclui a classe `Test::Unit::TestCase` (da qual a classe `TestEstado` é derivada).

Os testes a serem aplicados correspondem ao métodos de `TestEstado`, ou seja:

1. `test_grava_sigla_vazia`
2. `test_grava_nome_vazio`
3. `test_grava_nome_so_char`
4. `test_grava_nome_utf8`
5. `test_grava_nome_com_char_especial`

Ao executar este arquivo, o interpretador Ruby irá executar os testes em sequencia (no caso, do último para o primeiro).

Agora, vamos analisar o que ocorre em cada método. Como exemplo, considere o método `test_grava_sigla_vazia` (linha 6). O método é composto por comandos Ruby regulares onde cria-se um novo objeto e atribui `nil` à sigla. Espera-se que o modelo não permita a gravação deste registro, ou seja, que o comando `est.save` retorne `false`. A linha 11 indica que, se o retorno não for `false`, então há um erro (ou melhor, uma falha).

O funcionamento do método `assert` nem sempre é intuitivo. Em português, traduz-se como `afirmar` e apresenta várias variantes[7], todas herdadas de `Test::Unit::TestCase`.

Nesta explicação, utilizamos o método `assert_equal`, que utiliza três parâmetros. Ao usá-lo, afirma-se que a avaliação correta é que o primeiro parâmetro deve ser igual ao segundo parâmetro. A não igualdade entre eles implica numa falha (*failure*) e desencadeia a impressão da mensagem indicada no terceiro parâmetro.

Veja o método `test_grava_sigla_vazia`. Ele verifica se o modelo permite a gravação de um registro com sigla igual a `nil`. Na linha 10 é feita uma tentativa de gravação do registro. Se o modelo estiver funcionando corretamente, o resultado deve ser `"false"`. A linha seguinte é quem faz esta verificação, e pode ser lido da seguinte forma: afirmo que o resultado da operação (parâmetro 1) deve ser falso (parâmetro 2). Se não for, há uma falha no modelo e a mensagem do parâmetro 3 deve ser impressa.

A execução do arquivo `testeEstado.rb` imprime muitas linhas. Abaixo apresentamos como executá-lo, e nos concentramos nos resultado impresso sobre o método `test_grava_sigla_vazia`:

[7] Confira em `https://en.wikibooks.org/wiki/Ruby_Programming/Unit_testing#Available_Assertions`

```
require_relative "estadoSemValida"
require "test/unit"

class TestEstado < Test::Unit::TestCase

    def test_grava_sigla_vazia
        est = Estado.new
        est.nome = "Teste de Nome"
        est.sigla = nil
        res = est.save
        assert_equal(res,false,"Salvou com sigla vazia")
    end

    def test_grava_nome_vazio
        est = Estado.new
        est.sigla = "TS"
        est.nome = nil
        res = est.save
        assert_equal(res,false,"Salvou com nome vazio")
    end

    def test_grava_nome_so_char
        est = Estado.new
        est.sigla = "TS"
        est.nome = "ABCDEFGHIJKLMNOPQRSTUVXYZ"
        res = est.save
        assert_equal(res,true,"Não salvou com nome só com chars")
    end

    def test_grava_nome_utf8
        est = Estado.new
        est.sigla = "TS"
        est.nome = "ÁÉÍÓÚÀÃÕÂÊÔÇáãç"
        res = est.save
        assert_not_equal(res,false,"Não salvou com nome com UTF-8")
    end

    def test_grava_nome_com_char_especial
        char_especial = "!@#$%*()_-+=[{}]^ <,>.:;?'¨
        char_especial.each_char do |c|
            est = Estado.new
            est.sigla = "TS"
            puts "Testando gravação com #c"
            est.nome = c
            res = est.save
            assert_not_equal(res,true,"Salvou com nome com char especial")
        end
    end
end
```

Algoritmo 32 : Arquivo `"testeEstado.rb"`

```
> ruby testeEstado.rb
...
F
================================================================================
Failure:
test_grava_sigla_vazia(TestEstado)
testeEstado.rb:12:in ‘test_grava_sigla_vazia’
      9:     est.nome = "Teste de Nome"
     10:     est.sigla = nil
     11:     res = est.save
  => 12:     assert_equal(false,res,"Salvou com sigla vazia")
     13:   end
     14:
     15:   def test_grava_nome_vazio
Salvou com sigla vazia
<false> expected but was
<true>
...
Finished in 0.412276085 seconds.

5 tests, 5 assertions, 3 failures, 0 errors, 0 pendings, 0 omissions, 0 notifications
40% passed

12.13 tests/s, 12.13 assertions/s
```

Inicialmente, indica que foi detectada uma falha no método `test_grava_sigla_vazia`, mas especificamente na linha 12. Em seguida, imprime sete linhas centradas na linha 12. Por fim, imprime a mensagem indicada no assert e indica que era esperado `<false>`, mas encontrou `<true>`,

A parte final contém um resumo que começa com o tempo utilizado nos testes. Em seguida indica que foram executados cinco testes e cinco afirmações. Destas, três indicaram falhas.

Após analisar as falhas indicadas, o próximo passo é corrigi-las. Para a falha indicada pelo teste `test_grava_sigla_vazia` , basta incluir a validação necessária no modelo (`validates :sigla, presence: true` - veja seção 1.2.2) e executar novamente o arquivo de testes para obter um resultado semelhante a:

```
> ruby testeEstado.rb
...
Finished in 0.284432559 seconds.

5 tests, 5 assertions, 2 failures, 0 errors, 0 pendings, 0 omissions, 0 notifications
60% passed
```

Observe que o número de falhas reduziu de 3 para 2. Cabe ao programador corrigir as falhas e não incluir novas. Deixamos para o leitor a tarefa de eliminar os demais erros e também de elaborar outros testes que deixem o modelo mais robusto.

É importante destacar que o processo de elaboração de testes é uma ferramenta imprescindível para o desenvolvimento de software. Existem várias técnicas discutidas pela Engenharia de Software, e algumas foram implementadas em Ruby/Rails.

2. ActiveRecord em Rails

Este capítulo descreve como utilizar ActiveRecord em Rails. Todos os conceitos importantes já foram descritos no capítulo 1, em Ruby. Por esta razão, utilizaremos o mesmo roteiro lá apresentado, só que apontando como as coisas são mais fáceis em Rails.

Antes de começar, é importante destacar a forma de trabalho desta seção. Enquanto no capítulo 1 era necessário escrever programas do início, aqui o próprio Rails cria a maioria deles.

O primeiro passo é criar uma nova aplicação rails:

```
> rails new ARemRails
...
      create  app/models/.keep
      create  app/controllers/concerns/.keep
      create  app/models/concerns/.keep
...
      create  db
      create  db/seeds.rb
         run  bundle install
...
> cd ARemRails
```

O comando `rails new <diretório>` cria um diretório com o nome especificado (no caso, `ARemRails`). Este diretório é preenchido com vários arquivo que são listados, muitos deles ruby (extensão `.rb`). Como são muitos, não apresentamos todos, deixando só alguns importantes para o ActiveRecord.

O diretório `app/models/` é onde residem os modelos e o diretório `db` contém arquivos do banco de dados.

2.1 O básico: Activerecord em uma tabela

Esta seção reproduz em Rails o que foi feito na seção 1.1. Sugere-se fortemente já ter lido (e entendido) aquela seção antes daqui.

Uma das coisas mais legais no Rails é a forma de se criar uma aplicação completa a partir da linha de comando. Em uma linha é possível criar Modelos, Views e Controladores. É o caso do comando `rails generate scaffold`.

Porém, como o presente texto se concentra nos modelos e respectivas tabelas, utilizaremos outro comando, um que só cria os modelos, BD e tabelas.

```
> rails generate model Estado nome:string sigla:string{2}
     invoke  active_record
      create    db/migrate/20160616140449_create_estados.rb
      create    app/models/estado.rb
      invoke    test_unit
      create      test/models/estado_test.rb
      create      test/fixtures/estados.yml
```

Um modelo é composto por uma classe e uma tabela. Na seção 1.1.1, isto foi feito em dois arquivos:

`"estado.rb"` contém o modelo (veja algoritmo 3;

`"criaEstados.rb"` cria a tabela (veja algoritmo 2);

O comando `rails generate model` acima já criou estes dois arquivos:

`"app/models/estado.rb"` contém o modelo. Veja algoritmo 33 e compare com a versão Ruby.

`"db/migrate/20160616140449_create_estados.rb"` cria a tabela (veja algoritmo 34). O número que precede `create_estados.rb` será outro. Compare com a versão Ruby.

```
1 class Estado < ActiveRecord::Base;
2 end
```

Algoritmo 33 : Arquivo "app/models/estado.rb"

```
1  class CreateEstados < ActiveRecord::Migration
2      def change
3          create_table :estados do |t|
4              t.string :nome
5              t.string :sigla, limit: 2
6
7              t.timestamps null: false
8          end
9      end
10 end
```

Algoritmo 34 : Arquivo "db/migrate/20160616140449_create_estados.rb"

Os arquivos do diretório `db/migrate` são chamados arquivos de migração. Eles contém as informações sobre as tabelas a serem criadas e seus respectivos atributos. Porém, as tabelas só serão criadas após o comando `rake db:migrate`.

Enquanto não executa este comando, o programador pode editar as migrações, o que é comum na criação de relacionamentos. Como neste primeiro exemplo isto não é necessário:

```
> rake db:migrate
== 20160616140449 CreateEstados: migrating ====================================
-- create_table(:estados)
   -> 0.0314s
== 20160616140449 CreateEstados: migrated (0.0316s) ===========================
```

O efeito deste comando é a criação do banco de dados em `db/development.sqlite3`, que pode ser consultado como indicado abaixo:

```
sqlite3 db/development.sqlite3
SQLite version 3.8.2 2013-12-06 14:53:30
Enter ".help" for instructions
Enter SQL statements terminated with a ";"
sqlite> .tables
estados            schema_migrations
sqlite> .schema
CREATE TABLE "schema_migrations" ("version" varchar NOT NULL);
CREATE UNIQUE INDEX "unique_schema_migrations" ON "schema_migrations"
("version");
CREATE TABLE "estados" ("id" INTEGER PRIMARY KEY AUTOINCREMENT NOT
NULL, "nome" varchar, "sigla" varchar(2), "created_at" datetime NOT
NULL, "updated_at" datetime NOT NULL);
sqlite>
```

Comparando o procedimento descrito aqui para criação de modelo com Rails e o procedimento descrito na seção 1.1, pode-se afirmar que com Rails é necessário muito menos trabalho. É importante observar que Rails gera os arquivos automaticamente (ou como alguns preferem, automagicamente).

2.1.1 Operações

Executar operações de inclusão, alteração e exclusão na aplicação recém criada exige que a aplicação já esteja "levantada". Para tal será necessário utilizar dois terminais, ambos no diretório raiz da aplicação. No primeiro, digite `rails s`, que irá criar o ambiente Rails. Este terminal ficará travado após este comando.

Enquanto o primeiro terminal fica travado, inicie a execução do rails no segundo terminal conforme indicado abaixo.

```
> rails console --sandbox
Running via Spring preloader in process 19040
Loading development environment in sandbox (Rails 4.2.1)
Any modifications you make will be rolled back on exit
2.2.0 :001 >
```

O comando `rails console` permite a execução dos aplicativo em de forma interativa como exemplificado abaixo.

```
2.2.0 :001 > est = Estado.new
 => #<Estado id: nil, nome: nil, sigla: nil, created_at: nil, updated_at: nil>
2.2.0 :002 > est.nome="Acre"
 => "Acre"
2.2.0 :003 > est.sigla="AC"
 => "AC"
2.2.0 :004 > est.save
   (0.2ms)  SAVEPOINT active_record_1
  SQL (61.3ms)  INSERT INTO "estados" ("nome", "sigla", "created_at", "updated_at") VALUES (?, ?, ?, ?)  [["nome", "Acre"],
   (0.1ms)  RELEASE SAVEPOINT active_record_1
 => true
2.2.0 :005 > e = Estado.find_by_sigla("AC")
  Estado Load (0.4ms)  SELECT  "estados".* FROM "estados" WHERE "estados"."sigla" = ? LIMIT 1  [["sigla", "AC"]]
 => #<Estado id: 1, nome: "Acre", sigla: "AC", created_at: "2016-06-16 14:49:49", updated_at: "2016-06-16 14:49:49">
2.2.0 :006 > puts e.inspect
#<Estado id: 1, nome: "Acre", sigla: "AC", created_at: "2016-06-16 14:49:49", updated_at: "2016-06-16 14:49:49">
 => nil
2.2.0 :007 > estados = Estado.all
  Estado Load (0.2ms)  SELECT "estados".* FROM "estados"
 => #<ActiveRecord::Relation [#<Estado id: 1, nome: "Acre", sigla: "AC", created_at: "2016-06-16 14:49:49", updated_at: "201
2.2.0 :008 > estados.each do |e|
2.2.0 :009 >     puts e.inspect
2.2.0 :010?>   end
#<Estado id: 1, nome: "Acre", sigla: "AC", created_at: "2016-06-16 14:49:49", updated_at: "2016-06-16 14:49:49">
 => [#<Estado id: 1, nome: "Acre", sigla: "AC", created_at: "2016-06-16 14:49:49", updated_at: "2016-06-16 14:49:49">]
2.2.0 :011 > quit
 (01.1ms)  rollback transaction
```

Observe que todas as ações que acessaram as tabelas ecoaram os comandos SQL enviados ao SGBD (no caso, sqlite3).

Todos os programas apresentados em Ruby na seção 1.1 podem ser repassados aqui. Porém, é bastante incômodo inserir dados desta forma, principalmente sabendo que todas as operações executadas no console serão excluídas ao finalizá-la.

O ambiente rails permite a população de tabelas utilizando o comando `rake db:seed`. Este comando executa o arquivo `db/seeds.rb`.

Originalmente, seu conteúdo é como indicado no algoritmo 35. Observe que ele é um arquivo Ruby normal, e que os comandos sugeridos são equivalentes àqueles do arquivo `"populaEstados.rb"` (algoritmo 4 na página 12). Isto

```
1 # This file should contain all the record creation needed to seed the database with its
    default values.
2 # The data can then be loaded with the rake db:seed (or created alongside the db with
    db:setup).
3 #
4 # Examples:
5 #
6 # cities = City.create([ name: 'Chicago' ,  name: 'Copenhagen'])
7 # Mayor.create(name: 'Emanuel', city: cities.first)
```

Algoritmo 35 : Arquivo "db/seeds.rb"(original)

significa dizer que o este arquivo pode conter um programa Ruby qualquer. Por exemplo, um programa Ruby que popula uma tabela a partir de informações contidas num CSV, arquivo planilha, entre outros.

Comandos de alteração e exclusão seguem o mesmo modelo do que foi apresentado nas seções 1.1.2.4 e 1.1.2.3 e não serão abordados aqui.

2.2 Validações

Os mesmos mecanismos de validação descritos na seção 1.2 valem aqui: inclua as validações no modelo.

No caso de ser utilizado um arquivo de validação externo ao modelo (como sugerido com `validates_with` (seção 1.2.3), sugere-se criar um diretório específico, por exemplo `lib/validators`, e lá colocar todos os arquivos de validação. Porém, como este diretório não é padrão, o Rails não o incluirá ao levantar a aplicação. Para indicar que este diretório deve ser incluído, é necessário acrescentar o comando abaixo antes do penúltimo `end` do arquivo `"config/application.rb"`.

```
config.autoload_paths += %W(#{config.root}/lib/validators
```

Existem vários aspectos da linguagem Ruby que proporcionam a implementação de conceitos de engenharia de software. ActiveRecord é um exemplo de implementação do conceito de padrões aplicado ao acesso a bancos de dados.

Existem vários outros conceitos que foram implementados em Ruby, como por exemplo, conceitos de aplicação de testes - os ambientes de testes ou `frameworks` de testes.

Com o aparecimento de Rails, os aplicativos poderiam se tornar muito complexos e difíceis de testar, e surgiu a oportunidade de criar ambientes poderosos para testes. Surgiram várias implementações, algumas delas com propósito específico de aplicar determinado conceito (como por exemplo TDD[1]) e outros que mudam as técnicas que prometem ser mais robustas e poderosas que seus predecessores.

Para completar a seção de validações, as próximas seções explicam um destes `frameworks` de testes, aquele que já vem instalado como padrão.

2.3 *Callbacks*

O que foi apresentado na seção 1.3 pode ser aplicado em Rails.

2.4 Relacionamentos

Como a seção 1.4 já apresentou os modificadores utilizados pelo ActiveRecord, esta seção descreve objetivamente como utilizá-los para implementar as relações clássicas de bancos de dados (1:1, 1:N, N:N).

[1] test driven development

2.5 Relacionamento 1:1

Como já foi apresentado, a implementação do relacionamento 1:1 exige a colocação do modificador `belongs_to` em um modelo (aquele onde reside a chave estrangeira) e do modificador `has_one` no outro modelo.

O exemplo será o mesmo já visto anteriormente: pessoas (contém o modificador `belongs_to` no modelo e consequentemente a chave estrangeira na tabela) e documentos (só contém o modificador `has_one`. Como toda a parte conceitual já foi vista, será apresentado como criar os modelos e como utilizar os benefícios dos modificadores.

Documentos Cria a classe (`app/models/documento.rb`) e o arquivo de migração (`db/migrate/20160616215447_create_documentos.rb`)

```
rails g model Documento rg:string cpf:string motorista:string pessoa:belongs_to
Running via Spring preloader in process 3731
      invoke  active_record
      create    db/migrate/20160616215447_create_documentos.rb
      create    app/models/documento.rb
      invoke    test_unit
      create      test/models/documento_test.rb
      create      test/fixtures/documentos.yml
```

Pessoas Cria a classe (`app/models/pessoa.rb`) e o arquivo de migração (`db/migrate/20160616215459_create_pessoas.rb`)

```
> rails g model Pessoa nome:string endereco:string
Running via Spring preloader in process 3742
      invoke  active_record
      create    db/migrate/20160616215459_create_pessoas.rb
      create    app/models/pessoa.rb
      invoke    test_unit
      create      test/models/pessoa_test.rb
      create      test/fixtures/pessoas.yml
```

cria tabelas

```
> rake db:migrate
== 20160616215447 CreateDocumentos: migrating ================================
-- create_table(:documentos)
   -> 0.0243s
== 20160616215447 CreateDocumentos: migrated (0.0244s) =======================

== 20160616215459 CreatePessoas: migrating ===================================
-- create_table(:pessoas)
   -> 0.0343s
== 20160616215459 CreatePessoas: migrated (0.0345s) ==========================
```

Observe que na criação do modelo `Documento` já foi indicado o atributo `pessoa` cujo "tipo" é `belongs_to`. Esta facilidade foi incluída no comando para simplificar a criação da chave estrangeira de pessoas (veja no arquivo de migração), e já coloca o modificador `belongs_to` no modelo Documentos (confira).

Outras abordagens válidas:

- substituir o `belongs_to` por integer;
- não incluir o atributo `pessoa` na linha de comando e incluí-lo no arquivo de migração antes de criar as tabelas.

Com isto, o modelo e tabelas do lado que tem a chave estrangeira (Documento) está resolvido. No outro modelo (`app/model/pessoa.rb`) precisa acrescentar o modificador `has_one` (algoritmo 36)

Após "tudo" isso feito já é possível verificar se funciona utilizando o console.

```
1 class Pessoa < ActiveRecord::Base
2     has_one :documento
3 end
```

Algoritmo 36 : Arquivo app/models/pessoa.rb

```
>> d=Documento.new(:rg=>"01112223", :cpf=>"11122233344")
=> #<Documento id: nil, rg: "01112223", cpf: "11122233344", motorista: nil, pessoa_id: nil, created_at: nil, updated_a
>> d.save
   (0.2ms)  begin transaction
  SQL (29.0ms)  INSERT INTO "documentos" ("rg", "cpf", "created_at", "updated_at") VALUES (?, ?, ?, ?)  [["rg", "0111
   (43.9ms)  commit transaction
=> true
>> p = Pessoa.new(:nome=>"A", :documento=>d)
=> #<Pessoa id: nil, nome: "A", endereco: nil, created_at: nil, updated_at: nil>
>> p.save
   (0.2ms)  begin transaction
  SQL (42.5ms)  INSERT INTO "pessoas" ("nome", "created_at", "updated_at") VALUES (?, ?, ?)  [["nome", "A"], ["create
  SQL (0.8ms)  UPDATE "documentos" SET "pessoa_id" = ?, "updated_at" = ? WHERE "documentos"."id" = ?  [["pessoa_id", 
   (28.0ms)  commit transaction
=> true
>>  p.documento.rg
=> "01112223"
>>  d.pessoa.nome
=> "A"
>>
```

2.6 Relacionamento 1:N

A seção 1.4.3 apresentou o modificador has_many, utilizado para implementar a relação 1:n.

Como exemplo, foi utilizada a relação entre estados e municípios, onde um estado tem vários municípios e cada município está em um único estado.

Conforme explicado na seção 1.4.3, um dos dois modelos conterá o modificador belongs_to enquanto o outro conterá o modificador has_many.

Estados Já foi criada. Porém, a título de uma explicação mais completa, suponha que não foi. Assim, cria-se a classe Estado (app/models/estado.rb) e o arquivo de migração (db/migrate/20160616215447_create_documentos.rb)

```
> rails generate model Estado nome:string sigla:string{2}
invoke active_record
create
db/migrate/20160616140449_create_estados.rb
create
app/models/estado.rb
invoke
test_unit
create
test/models/estado_est.rb
create
test/fixtures/estados.yml
```

Município Cria a classe (app/models/municipio.rb) e o arquivo de migração (db/migrate/20160616215459_create_municipios.rb). Precisa acrescentar o has_many no modelo.

```
> rails g model Municipio nome:string estado:belongs_to
Running via Spring preloader in process 5966
      invoke  active_record
      create    db/migrate/20160616234204_create_municipios.rb
      create    app/models/municipio.rb
      invoke    test_unit
      create      test/models/municipio_test.rb
      create      test/fixtures/municipios.yml
```

cria tabelas

```
> rake db:migrate
== 20160616234204 CreateMunicipios: migrating ================================
-- create_table(:municipios)
   -> 0.2214s
== 20160616234204 CreateMunicipios: migrated (0.2216s) ========
```

Após "tudo" isso feito já é possível verificar se funciona utilizando o console.

```
>> est = Estado.find_by_sigla("PR")
 Estado Load (7.9ms)  SELECT  "estados".* FROM "estados" WHERE "estados"."sigla" = ? LIMIT 1  [["sigla", "PR"]]
=> #<Estado id: 14, nome: "Paraná", sigla: "PR", created_at:
"2016-06-16 23:45:34", updated_at: "2016-06-16 23:45:34">
>> munic = Municipio.new(:nome=>"Abatia",:estado=>est)
=> #<Municipio id: nil, nome: "Abatia", estado_id: 14, created_at: nil, updated_at: nil>
>> munic.save
   (0.2ms)  SAVEPOINT active_record_1
  SQL (19.2ms)  INSERT INTO "municipios" ("nome", "estado_id", "created_at", "updated_at") VALUES (?, ?, ?, ?)  [["nome", "A
   (0.1ms)  RELEASE SAVEPOINT active_record_1
=> true
>> munic = Municipio.new(:nome=>"Adrianópolis",:estado=>est)
=> #<Municipio id: nil, nome: "Adrianópolis", estado_id: 14, created_at: nil, updated_at: nil>
>> munic.save
   (0.1ms)  SAVEPOINT active_record_1
  SQL (0.2ms)  INSERT INTO "municipios" ("nome", "estado_id", "created_at", "updated_at") VALUES (?, ?, ?, ?)  [["nome", "Ad
   (0.1ms)  RELEASE SAVEPOINT active_record_1
=> true
>> est_pr = Estado.find_by_sigla("PR")
  Estado Load (0.1ms)  SELECT  "estados".* FROM "estados" WHERE "estados"."sigla" = ? LIMIT 1  [["sigla", "PR"]]
=> #<Estado id: 14, nome: "Paraná", sigla: "PR", created_at: "2016-06-16 23:45:34", updated_at: "2016-06-16 23:45:34">
>>  puts est_pr.nome
Paraná
=> nil
>> est_pr.municipios.each do |m| puts m.nome end
  Municipio Load (0.3ms)  SELECT "municipios".* FROM "municipios" WHERE "municipios"."estado_id" = ?  [["estado_id", 14]]
Abatia
Adrianópolis
=> [#<Municipio id: 5, nome: "Abatia", estado_id: 14, created_at: "2016-06-17 00:15:28", updated_at: "2016-06-17 00:15:28">,
>>
```

2.7 Relacionamento N:N

A seção 1.4.4 descreveu como implementar uma relação N:N. Resumidamente temos que:

1. criar dois modelos (Pessoas e Esportes) incluindo o modificador `has_and_belongs_to_many` em cada um deles, e o parâmetro do modificador será o outro modelo.
2. criar uma tabela tabela de referência, que na padronização do ActiveRecord se chamará `esportes_pessoas`. **Importante:** na ordem alfabética, ou seja `pessoas_esportes` não funciona!

Pessoas Tabela já criada, mas a título uma explicação mais completa, suponha que não foi. Assim, cria-se a classe Pessoa (`app/models/pessoa.rb` e o arquivo de migração. Precisa acrescentar o `has_and_belongs_to_many` no modelo (veja algoritmo 38)

```
> rails g model Pessoa nome:string endereco:string
Running via Spring preloader in process 4970
      invoke  active_record
      create    db/migrate/20160617164142_create_pessoas.rb
      create    app/models/pessoa.rb
      invoke    test_unit
      create      test/models/pessoa_test.rb
      create      test/fixtures/pessoas.yml
```

Esportes Cria a classe (`app/models/esporte.rb`) e o arquivo de migração (`db/migrate/20160617162101_create_esportes.rb`). Precisa acrescentar o `has_and_belongs_to_many` no modelo veja algoritmo 37.

```
> rails g model Esporte nome:string
Running via Spring preloader in process 4194

      invoke  active_record
      create    db/migrate/20160617162101_create_esportes.rb
      create    app/models/esporte.rb
      invoke    test_unit
      create      test/models/esporte_test.rb
      create      test/fixtures/esportes.yml
```

Tabela de Referência (esportes_pessoas) A ordem importa: pessoas_esportes não funciona!

```
> rails g migration CreateJoinTableEsportePessoa esporte pessoa
Running via Spring preloader in process 5628
      invoke  active_record
      create    db/migrate/20160617170721_create_join_table_esporte_pessoa.rb
```

Executa as migrações

```
> rake db:migrate
== 20160617170710 CreatePessoas: migrating ====================================
-- create_table(:pessoas)
   -> 0.0025s
== 20160617170710 CreatePessoas: migrated (0.0027s) ===========================

== 20160617170714 CreateEsportes: migrating ===================================
-- create_table(:esportes)
   -> 0.0011s
== 20160617170714 CreateEsportes: migrated (0.0012s) ==========================

== 20160617170721 CreateJoinTableEsportePessoa: migrating =====================
-- create_join_table(:esportes, :pessoas)
```

```
   -> 0.0017s
== 20160617170721 CreateJoinTableEsportePessoa: migrated (0.0019s) ============
```

```
1 class Esporte < ActiveRecord::Base
2     has_and_belongs_to_many :pessoas
3 end
```

Algoritmo 37 : Arquivo `app/models/esporte.rb`

```
1 class Pessoas < ActiveRecord::Base
2     has_and_belongs_to_many :esportes
3 end
```

Algoritmo 38 : Arquivo `app/models/pessoa.rb`

Após estes passos, já é possível aplicar os métodos obtidos com o uso do modificador (veja seção 1.4.4.1) no console:

```
> e = Esporte.new(:nome => "futebol")
=> #<Esporte id: nil, nome: "futebol", created_at: nil, updated_at: nil>
> e.save
   (0.2ms)  SAVEPOINT active_record_1
  SQL (0.6ms)  INSERT INTO "esportes" ("nome", "created_at", "updated_at") VALUES (?, ?, ?)  [["nome", "futebol"], ["created
   (0.1ms)  RELEASE SAVEPOINT active_record_1
> p = Pessoa.new(:nome => "Pedro")
=> #<Pessoa id: nil, nome: "Pedro", endereco: nil, created_at: nil, updated_at: nil>
> p.save
  (0.2ms)  SAVEPOINT active_record_1
  SQL (0.4ms)  INSERT INTO "pessoas" ("nome", "created_at", "updated_at") VALUES (?, ?, ?)  [["nome", "Pedro"], ["created_at
   (0.1ms)  RELEASE SAVEPOINT active_record_1
> futebol = Esporte.find_by_nome("futebol")
 Esporte Load (0.3ms)  SELECT  "esportes".* FROM "esportes" WHERE "esportes"."nome" = ? LIMIT 1  [["nome", "futebol"]]
=> #<Esporte id: 1, nome: "futebol", created_at: "2016-06-17 16:49:03", updated_at: "2016-06-17 16:49:03">
> pedro = Pessoa.find_by_nome("Pedro")
  Pessoa Load (0.3ms)  SELECT  "pessoas".* FROM "pessoas" WHERE "pessoas"."nome" = ? LIMIT 1  [["nome", "Pedro"]]
=> #<Pessoa id: 1, nome: "Pedro", endereco: nil, created_at: "2016-06-17 16:50:27", updated_at: "2016-06-17 16:50:27">
> pedro.esportes << futebol
   (0.0ms)  SAVEPOINT active_record_1
  SQL (0.1ms)  INSERT INTO "esportes_pessoas" ("pessoa_id", "esporte_id") VALUES (?, ?)  [["pessoa_id", 1], ["esporte_id", 1
   (0.0ms)  RELEASE SAVEPOINT active_record_1
  Esporte Load (0.1ms)  SELECT "esportes".* FROM "esportes" INNER JOIN "esportes_pessoas" ON "esportes"."id" = "esportes_pes
=> #<ActiveRecord::Associations::CollectionProxy [#<Esporte id: 1, nome: "futebol", created_at: "2016-06-17 16:49:03", updat
> pedro.save
   (0.1ms)  SAVEPOINT active_record_1
   (0.1ms)  RELEASE SAVEPOINT active_record_1
```

Como exercício, implemente o algoritmo 30 para comprovar como funcionam os dois lados do relacionamento.

2.8 Testes

[2]

[2] https://www.caelum.com.br/apostila-ruby-on-rails/apendice-testes/#16-2-test-unit

Como uma das características da linguagem Ruby é a facilidade de criar `frameworks`, e foram implementadas várias gemas para automatizar os testes. Alguns, inclusive, utilizam métodos propostos de Engenharia de Software, em especial BDD[3], que são implementados em duas bibliotecas populares: `rspec`[4] e `cucumber`[5].

Como o objetivo é agilizar a compreensão do funcionamento de testes em Rails, será descrito o framework "nativo", aquele que é automaticamente incluído na aplicação. Uma vez compreendido como funciona este, é mais fácil compreender os demais.

De uma maneira geral pode-se dizer que o processo todo consiste em:

1. criar uma massa de dados para testes;
2. criar métodos contendo os testes a serem executados e que emitem um alerta quando os testes forem mal sucedidos.

A idéia geral é semelhante àquela descrita na seção 1.5, porém numa aplicação Rails, os arquivos de teste são separados em um diretório próprio.

Para exemplificar, vamos começar uma aplicação do zero, nela implementar um único modelo e aplicar testes e validações. Isto permite compartimentalizar a explicação.

Como na seção 1.5, o modelo que vamos implementar é Estados, e serão aplicados os mesmos testes. De acordo com a necessidade imposta pela validação (não pode ter nome ou sigla vazia, só pode conter caracteres, etc.), iremos criar testes (que inicialmente vão causar efeitos incorretos) e depois iremos implementar as respectivas validações.

O primeiro passo é criar a aplicação conforme especificado a seguir, onde está destacado o diretório de testes.

```
> rails new Testes
...
      create  test/fixtures
      create  test/fixtures/.keep
      create  test/controllers
      create  test/controllers/.keep
      create  test/mailers
      create  test/mailers/.keep
      create  test/models
      create  test/models/.keep
      create  test/helpers
      create  test/helpers/.keep
      create  test/integration
      create  test/integration/.keep
      create  test/test_helper.rb
...
```

Observe que existem vários diretórios: `controllers`, `fixtures`, `helpers`, `integration`, `mailers` e `models`. Cada um deles tem como objetivo testar um aspecto específico da aplicação. Não vamos citar todas, mas como exemplo o diretório `test/controllers` contém os testes que verificam se as views corretas serão renderizadas a cada requisição.

Como o foco deste livro é ActiveRecord, iremos descrever um único diretório, aquele que testa se os modelos são consistentes. Este módulo de testes é chamado *Unit Test of Models*, ou testes de modelo ou ainda testes de unidade.

Vamos agora criar um modelo para analisar o que o Rails já inclui para testar aquele modelo:

```
> rails generate model Estado nome:string sigla:string2
Running via Spring preloader in process 7034
      invoke  active_record
      create    db/migrate/20160619150154_create_estados.rb
      create    app/models/estado.rb
      invoke    test_unit
      create      test/models/estado_test.rb
```

[3] Behavior Driven Development
[4] http://rspec.info/
[5] https://cucumber.io/

```
      create      test/fixtures/estados.yml
> rake db:migrate
== 20160619150154 CreateEstados: migrating ===================================
-- create_table(:estados)
   -> 0.0223s
== 20160619150154 CreateEstados: migrated (0.0225s) ==========================
```

Observe que foram criados dois arquivos para o teste do modelo:

test/models/estado_test.rb Contém os métodos contendo os testes;

test/fixtures/estados.yml Contém a massa de dados para a execução dos testes;

Também observe que o último comando contém a migração, pois a execução dos testes exige que as tabelas já tenham sido criadas.

Para executar o ambiente de testes, utiliza-se o comando abaixo:

```
> rake test
Running via Spring preloader in process 7170
Run options: --seed 24248

# Running:
Finished in 0.001957s, 0.0000 runs/s, 0.0000 assertions/s.

0 runs, 0 assertions, 0 failures, 0 errors, 0 skips
```

Não há surpresa nenhuma com o fato de não ter ocorrido nenhum erro, mas o que importa é saber o que o comando `rake test` fez:

2.8.1 O que ocorre com `rake test`

Cada vez que o comando `rake test` é disparado, os seguintes passos são executados [3]:

1. inicia o banco de dados de teste;
2. para cada arquivo de cada diretório de testes, faça:
   - para cada teste contido no arquivo, faça:
     a) executa método `setup`;
     b) executa os métodos de teste;
     c) executa método `teardown`;
     d) limpa banco de dados;

O passo 1 é iniciar o banco de dados. O nome do BD de teste é especificado no arquivo `config/database.yml` (confira).

O passo 2 percorre todos os diretórios de testes contidos em `test`, que são: `controllers`, `helpers`], `integration`], `mailers` e `models`.

O alvo deste texto é unicamente o último (`models`). Neste caso, consideramos que o passo 2 seleciona somente os arquivos contidos em `test/models`. Neste momento, só há um arquivo lá: `estado_test.rb` (que é igual ao apresentado no algoritmo 39);

```
1 require 'test_helper'
2
3 class EstadoTest < ActiveSupport::TestCase # test "the truth" do # assert true # end end
```

Algoritmo 39 : Conteúdo original do arquivo `estado_test.rb`

O passo 2a executa o método `setup`. Este método é herdado de `ActiveSupport::TestCase`, mas pode ser criado neste arquivo, substituindo o original.

O passo 2b executa cada um dos métodos contidos no arquivo, mas há uma pequena diferença sintática: enquanto os métodos de Ruby são definidos pelas palavras reservadas `def metodo-end`, os métodos de teste são definidos pelas palavras reservadas `test "teste"do-end`[6].

O passo 2c executa o método `teardown`. Como no passo 2a este método é herdado de `ActiveSupport::TestCase`, mas pode ser criado neste arquivo, substituindo o original.

Finalmente, o passo 2d limpa o BD dos registros incluídos para teste.

2.8.2 O ciclo do programador para testar um modelo

A seção 1.5 (página 37) descreveu como testar um modelo em Ruby. Apesar de só se referir ao modelo, ele pode ser aplicado a outras classes.

Em Rails, a codificação dos testes é semelhante, porém há um diretório específico só para os arquivos de testes, o diretório `test` na raiz da aplicação. Este diretório, por sua vez é dividido em outros, dos quais destacamos[7]:

fixtures dados a serem usados nos testes;

controllers testa se os controladores tratam as mensagens recebidas de forma correta e se geram as respostas apropriadas (por exemplo, views html).

mailers testa a classe de emails para verificar se os emails são criados e enviados corretamente, se o conteúdo é o correto e são enviados no momento correto;

models testa os modelos;

helpers testa os helpers desenvolvidos para as views da aplicação;

integration testa a interação entre as várias partes da aplicação;

Como a regra geral em Rails, existe muito material na Web que explica detalhadamente cada um dos tópicos acima. Este texto concentra-se nos diretórios `fixtures` e `models`.

Após criar a aplicação e o modelo, o diretório de testes de modelo já contém os arquivos de teste, um arquivo de teste por modelo.

Da mesma forma foi criado um arquivo no diretório `fixtures` contendo a massa de testes. Originalmente, o arquivo é semelhante ao apresentado no algoritmo 40

```
1  # Read about fixtures at
2  http://api.rubyonrails.org/classes/ActiveRecord/FixtureSet.html
3
4  one:
5      nome: MyString
6      sigla: MyString
7
8  two:
9      nome: MyString
10     sigla: MyString
11
```

Algoritmo 40 : Conteúdo original do arquivo `test/fixtures/estado.yml`

Este arquivo está no formato `yml`[8], que é simples e intuitivo: em um nível, há o nome de cada registro (no caso, `one` e `two`). Num segundo nível descreve o conteúdo de cada registro indicando os atributos do modelo estado (`nome` e `sigla`).

Para ilustrar o uso do arquivo de *fixtures*, iremos incluir lá os mesmos dados utilizados na seção 1.5, algoritmo 32. O resultado pode ser visto no algoritmo 42.

6 que significa algo como `def teste - end`

7 `http://guides.rubyonrails.org/testing.html`

8 abreviação de `yaml`: *yet another meta language*

```
1  # Read about fixtures at
2  http://api.rubyonrails.org/classes/ActiveRecord/FixtureSet.html
3
4  sigla_vazia:
5      nome ="Teste de Nome"
6      sigla =
7
8  nome_vazio:
9      sigla = "TS"
10     nome =
11
12 nome_char:
13     sigla = "TS"
14     nome = "ABCDEFGHIJKLMNOPQRSTUVXYZ"
15
16 nome_utf8:
17     sigla = "TS"
18     nome = "ÁÉÍÓÚÀÃÕÂÊÔÇáãç"
```

Algoritmo 41 : Nova versão do arquivo `test/fixtures/estado.yml`

```
1  require 'test_helper'
2
3  class EstadoTest < ActiveSupport::TestCase
4      test "Salvou com sigla vazia"do
5          estado = estados(:sigla_vazia)
6          assert_equal(estado.save, false)
7      end
8
9      test "Salvou com nome vazio"do
10         estado = estados(:nome_vazio)
11         assert_equal(estado.save,false)
12     end
13 end
```

Algoritmo 42 : Nova versão do arquivo `test/fixtures/estado.yml`

Observe o teste `"Salvou com sigla vazia"` (linhas 4 até). A primeira ação é criar um novo estado com a entrada `:sigla_vazia` do arquivo de `fixtures`. Em seguida (linha 6), aplica-se o método `assert`.

Ao executar este teste, obtém-se o seguinte resultado

```
> rake test
Run options: --seed 23851

# Running:

FF

Finished in 0.302060s, 6.6212 runs/s, 6.6212 assertions/s.

  1) Failure:
EstadoTest#test_Salvou_com_sigla_vazia [/home/bmuller/fradim/Ruby2/ActiveR/Texto/Testes/test/models/estado_test.rb:8]:
Expected: true
  Actual: false
```

```
  2) Failure:
EstadoTest#test_Salvou_com_nome_vazio [/home/bmuller/fradim/Ruby2/ActiveR/Texto/Testes/test/models/estado_test.rb:13]
Expected: true
  Actual: false

2 runs, 2 assertions, 2 failures, 0 errors, 0 skips
```

A execução dos testes deixa claro que houveram duas falhas e o local delas (linhas 8 e 13).

Ao incluir a validação na classe de estados (algoritmo 43, uma nova execução não acusará falhas.

```
1 class Estado < ActiveRecord::Base
2       validates :sigla, presence: true
3       validates :nome, presence: true
4 end
```

Algoritmo 43 : Modelo de estados com validação

2.8.3 Mais sobre testes

Este texto tratou somente de testes de modelo, mas existem muitos outros a estudar. Um sítio que trata o tema de forma completa é `http://guides.rubyonrails.org/testing.html#testing-helpers`. Além dele existem muitos outros.

Os livros que ensinam Rails normalmente tratam do tema, mas tão superficialmente (e magicamente) que chega a atrapalhar. Para um tratado completo e aprofundado de testes, sugiro [3].

3. Métodos Frequentemente Utiizados

3.1 Métodos de busca

3.1.1 all

```
pessoas=Pessoa.all
pessoas.each do |p|
   ...
end
```

3.1.2 find_by

Seleciona um registro que atende os parâmetros.

```
Pessoa.find_by(nome: "Pedro")
Estado.find_by(nome: "Acre", sigla: "AC")
```

A notação abaixo também vale.

```
Pessoa.find_by_nome("Pedro")
Estado.find_by_nome_and_sigla("Acre", "AC")
```

Eu prefiro a primeira notação por que o nome do atributo estar mais próximo do parâmetro.

3.1.3 where

Seleciona um conjunto de registros que atendem os parâmetros.

```
Pessoa.where(nome: "Pedro")
Estado.where(nome: "Acre", sigla: "AC")
# Nome contém substring "AR" em caixa alta
Estado.where("nome like ?", "%AR%")
# Nome contém substring "ar" independente de caixa
Estado.where("lower(nome) like ?", "%ar.downcase%").

Pessoa.where("idade > 20")

pessoas=Pessoa.all
p = pessoas.where("idade > 20")
```

3.1.4 order

Aplivável em um conjunto de registros, ordena de acordo com os parâmetros.

```
Pessoa.where(nome: "Pedro").order(:nome)
Estado.where(nome: "Acre", sigla: "AC").order(:sigla)
```

3.1.5 select

Indica quais atributos são requisitados na resposta.

```
Pessoa.where(nome: "Pedro").select(:nome)
```

3.1.6 delete

Apaga o registro indicado.

```
Pessoa.find_by(nome: "Pedro").delete
```

3.1.7 delete_all

Apaga todos os registros da tabela.

```
Pessoa.delete_all
```

3.2 Métodos de iteração

3.2.1 find_each

```
Pessoa.all.find_each do |p|
   ...
end

Pessoa.where("idade > 20").find_each do |p|
   ...
end
```

3.2.2 find_in_batches

Ao invés de trazer todos os registros para a memória de uma só vez, traz grupos (*batches*) e trata cada grupo em separado. Útil em paralelismo.

https://api.rubyonrails.org/classes/ActiveRecord/Batches.html

```
Person.where("age > 21").find_in_batches do |group|
  sleep(50) # Make sure it doesn't get too crowded in there!
  group.each { |person| person.party_all_night! }
end
```

Referências

[1] W. Jason Gilmore. *Easy Active Record for Rails Developers*. W.J. Gilmore, LLC, 2014.

[2] Martin Fowler, David Rice, Matthew Foemmel, Edward Hieatt, and Robert Mee. *Patterns of Enterprise Application Architecture*. Addison Wesley, 2002.

[3] Noel Rappin. *Rails Test Prescriptions*. Pragmatic Programmers, LLC., 2010.